# Liquid Gated Attention

Yiheng Jiang, Yuanbo Xu, and Yongjian Yang

arXiv:2608.30695v1 [cs.LG] 31 Aug 2026

***Abstract*—Real-world time series often exhibit irregular sampling and extended temporal horizons, requiring models to capture continuous-time dynamics across arbitrary time intervals without prohibitive scaling costs. Conventional discrete-time methods typically project observations onto uniformly spaced indices, collapsing variable time intervals into static positional steps and discarding temporal distances between observations. Conversely, solver-dependent continuous-time models preserve temporal structure but rely on sequential numerical integration, precluding parallelization. While solver-free approximations bypass this computational overhead, no parallel architecture explicitly couples observed time intervals with input-driven state modulation, limiting their ability to distinguish genuine temporal dynamics from observational noise. To bridge this gap, we propose Liquid Gated Attention (LGA), a solver-free parallel temporal operator. By parameterizing an input-driven gating mechanism with observed time intervals, LGA introduces a continuous-time inductive bias and formulates hidden state evolution as a fast-weight associative memory, enabling parallel computation across the temporal dimension. By leveraging matrix associativity in non-causal encoding and a prefix scan in causal encoding, LGA achieves linear temporal complexity with respect to sequence length in both modes. To stabilize long-horizon optimization, we employ a sequence-level normalization that bounds the cumulative temporal decay. Building on this operator, we instantiate LFormer, a modular backbone for continuous-time representation learning. Evaluations across six tasks and sixteen datasets, spanning up to 17,984 steps, demonstrate LFormer's capabilities in long-range dependency modeling, fine-grained state tracking, and trajectory reconstruction from sparse and noisy observations. LFormer delivers competitive performance against state-of-the-art discrete-time and continuous-time baselines, alongside linear scaling efficiency confirmed by empirical benchmarks. These results illustrate how LGA embeds continuous-time dynamical principles into a parallel architecture, positioning LFormer as a scalable and versatile backbone for time series representation learning. Code is available at https://anonymous.4open.science/r/Liquid-Gated-Attention-6B55.**

***Index Terms*—Continuous-time Model, Linear Attention, Time Series Analysis, Numerical Stability**

## I. Introduction

MONITORING real-world processes, such as physiological states [1], financial fluctuations [2], and biological signaling [3], typically yields complex time series that exhibit irregular sampling and can span extended temporal horizons. These properties present significant challenges for temporal modeling. Specifically, irregular sampling requires models to characterize continuous-time dynamics across arbitrary observed time intervals [4], [5]. Simultaneously, extended sequence lengths demand capturing long-range dependencies without prohibitive computational costs [6], [7]. Furthermore, realistic observational constraints call for algorithms that extract the underlying continuous-time process from sparse and noisy measurements [8], [9].

Most existing approaches operate within the discrete-time paradigm that projects continuous temporal trajectories onto uniformly spaced indices [10]. Despite excelling at handling regularly sampled data, this paradigm collapses variable intervals into static positional steps and discards the temporal distances between observations, which are necessary for accurate continuous-time dynamics modeling [4], [5], [10]. Recurrent neural networks (RNNs) [11], [12] and Transformers [13] represent two dominant discrete-time models. Standard RNNs compress temporal dependencies into sequential hidden states. To accommodate irregular observations, they typically rely on heuristic preprocessing strategies, such as binning [14] and imputation [15], which inject spurious states into the timeline and distorts the underlying continuous dynamics [5]. Moreover, their recurrence precludes parallelization along the temporal dimension. Transformers eliminate this sequential constraint through parallel self-attention. Yet, even when augmented with explicit temporal encoding [16]–[18], they collapse time intervals into static similarity weights between discrete tokens, lacking a dedicated mathematical mechanism to model continuous-time dynamics between observations [10]. Furthermore, their quadratic complexity limits scalability to extended temporal horizons.

To preserve temporal structure, continuous-time approaches formulate the state evolution as a differential equation parameterized by a neural network. Neural ordinary differential equations pioneered this formulation [19]. Subsequent extensions, including neural controlled and rough differential equations, accommodate irregularly sampled time series through continuous path integration [20], [21]. Rooted in differential calculus, these methods explicitly incorporate temporal distances between observations [5] and can naturally smooth high-frequency noise [8]. However, their forward pass relies on numerical solvers for sequential integration. This solver-dependent design introduces two bottlenecks. First, numerical solvers require multiple function evaluations per time step, and their sequential execution resists parallelization [22]. This computational overhead challenges these methods in large-scale or real-time deployments. Second, truncation errors can accumulate over extended integration horizons [23], progressively degrading the fidelity of learned dynamics.

These bottlenecks motivate solver-free alternatives that retain continuous-time dynamics while bypassing iterative integration. To render continuous state evolution directly evaluable, three principal strategies have emerged: closed-form approximations [22], linear state-space formulations [24], and truncated algebraic path signatures [25]. Despite eliminating the solver-induced computational overhead, these meth-

All authors are with the Mobile Intelligent Computing Laboratory, College of Computer Science and Technology, Jilin University, China (e-mail: jiangyh22@mails.jlu.edu.cn; yuanbox@jlu.edu.cn; yyj@jlu.edu.cn).

Corresponding author: Yuanbo Xu.

ods commit continuous-time dynamics to fixed mathematical forms. These compromises introduce specific trade-offs documented in their respective literature, including potential gradient decay in closed-form recurrence [22], constrained expressivity for nonlinear dynamics in linear state transitions [6], and information loss from truncated signatures [25]. Furthermore, within this paradigm, no parallel architecture explicitly couples observed time intervals with input-driven state modulation. Without this linkage, they struggle to distinguish noise from genuine state transitions [5], [9], which limits their robustness under sparse and noisy conditions.

Across these three paradigms, a critical gap emerges: no single approach simultaneously incorporates observed time intervals, enables parallel computation, and maintains observational noise robustness. Discrete-time models discard the underlying temporal metric. Solver-dependent continuous-time methods retain temporal fidelity and noise robustness but sacrifice parallelism. Solver-free approaches recover parallel scalability yet lack an explicit temporal-state transition link, compromising robustness to sparse and noisy measurements.

In this work, we address this trilemma via **L**iquid **G**ated **A**ttention (LGA), a solver-free temporal operator reconciling observed time intervals, parallel computation, and observational noise robustness across four design steps:

**(1) Continuous-time Gating.** LGA derives its gating mechanism from the closed-form structure of a one-dimensional liquid time-constant (1D LTC) equation [26]. The resulting **liquid gate** jointly encodes temporal decay and input-driven modulation, providing a continuous-time inductive bias that attenuates high-frequency noise perturbations.

**(2) Computational Efficiency.** To avoid the computational overhead of numerical integration, we approximate the liquid gate via a learnable endpoint interpolation inspired by the trapezoidal rule. Fixing the interpolation weight at $0.5$ recovers the classical trapezoidal rule with second-order local accuracy, whereas the generalized learnable configuration serves as a trainable numerical surrogate. This step parallelizes the gating calculation while preserving the continuous-time inductive bias.

**(3) Expressive Capability.** To capture complex long-range dependencies, we elevate the scalar state in 1D LTCs to a matrix-valued associative memory. Adopting a fast-weight programming framework [27], we translate the liquid gate into explicit modulation coefficients for attention features. This design preserves the continuous-time gating dynamics while yielding a parallel sequence operator that maintains linear temporal complexity with respect to sequence length in both non-causal and causal modes, the latter realized via a prefix scan.

**(4) Numerical Safety.** To stabilize long-horizon optimization, we introduce a sequence-level normalization that bounds the cumulative decay coefficients, preventing exponential underflow. This normalization operates as a regularized surrogate rather than an algebraically exact equivalent of the original LTC decay: it bounds the total decay budget while preserving the relative temporal distance allocation across the sequence.

Building upon the LGA operator, we instantiate **LFormer**, a modular backbone for continuous-time representation learning. LFormer constructs its deep temporal representations by interleaving decoupled multi-head LGA layers with channel-mixing blocks, connected via residual connections with layer normalization. LFormer directly processes discrete observations, while inheriting the continuous-time inductive bias, noise robustness, linear temporal complexity, and training stability properties from the underlying LGA operator.

This work delivers three primary contributions:

- **A parallel continuous-time temporal operator.** LGA unifies the input-driven and time-interval-coupled liquid gate, a trapezoidal-rule-inspired numerical integral surrogate, fast-weight associative memory, and sequence-level normalization into a single solver-free formulation.
- **A modular backbone for continuous-time representation learning.** LFormer instantiates LGA in a decoupled multi-head form within a residual architecture. This design enables linear-complexity parallel sequence modeling directly over discrete observations, while inheriting LGA's continuous-time inductive bias, noise robustness, and training stability properties.
- **A comprehensive empirical validation across diverse temporal scenarios.** Extensive evaluations across six distinct tasks and sixteen datasets demonstrate that LFormer achieves competitive performance compared to state-of-the-art discrete and continuous baselines. These outcomes highlight its capabilities in long-range dependency modeling, fine-grained state tracking, and trajectory reconstruction from sparse and noisy observations, while maintaining linear scaling efficiency and empirical training stability over extended temporal horizons.

Ultimately, these results illustrate how LGA embeds continuous-time dynamical principles into a parallel architecture, positioning LFormer as a scalable and versatile backbone for time series representation learning.

## II. Related Work

This section reviews prior literature along two axes. We first examine three temporal modeling paradigms distinguished by their mathematical treatment of time. We then discuss scalable parallelization mechanisms, focusing on fast-weight programmers and linear attention variants, which establish the computational foundation of the proposed architecture.

### A. Paradigms in Temporal Modeling

*1) Discrete-time Models:* This paradigm projects continuous temporal trajectories onto uniformly spaced discrete indices. Early approaches partition timeline into consecutive uniform bins [14], [28]–[30], and subsequently leverage imputation strategies [15], [31]–[33] to fill the resulting grids, yielding regularly sampled inputs on which standard architectures like RNNs [11], [12] and Transformers [13] achieve strong performance. However, these heuristic preprocessing strategies introduce pseudo-observations that can alter the underlying continuous dynamics [4], [34]. Recent methods shift this discrete-time paradigm to the sequence level by

grouping adjacent time steps into rigid patches [35]–[39]. Although patching optimizes effective sequence lengths and mitigates the self-attention quadratic complexity, these rigid boundary partitions compromise fine-grained temporal structure. Alternatively, several approaches enrich token representations with explicit temporal encoding, including sinusoidal functions [16], [17], kernel-based methods [40], and learnable embeddings [18], [41]. Nevertheless, these encodings reduce time to an auxiliary feature attached to discrete tokens, rather than leveraging it as a continuous variable that could inform state transitions. Consequently, they offer limited capacity for capturing input-dependent continuous-time dynamics [10].

*2) Solver-dependent Continuous-time Models:* In contrast to discrete-time approaches, this paradigm parameterizes the derivative of continuous states via neural networks, solving the resulting differential equations through numerical methods [8]. This formulation affords a mathematically natural description of continuous-time dynamics. Pioneered by neural ordinary differential equations [19], this formulation underpins a broad lineage of architectures. Latent ODEs [5] extend this framework to generative modeling for irregularly sampled time series. Neural controlled differential equations [20] and neural rough differential equations [21] construct continuous input paths to drive continuous-time state evolution. Stochastic extensions [42] model uncertainty within these latent processes, while continuous-time attention variants [10], [43] integrate differential equations with the self-attention mechanism. Although mathematically principled, this solver-dependent paradigm incurs substantial computational overhead, particularly when integrating stiff equations [22]. Furthermore, sequential numerical integration inherently limits parallelization [44]. Moreover, truncation errors can accumulate over extended integration horizons [23], progressively degrading the fidelity of the learned continuous-time dynamics.

*3) Solver-free Continuous-time Models:* To circumvent the computational bottlenecks of numerical integration, this closely related paradigm transforms continuous-time dynamics into directly evaluable algebraic forms. Three principal strategies are prevalent. First, closed-form approximations solve specific differential equations analytically, as exemplified by closed-form continuous-time (CfC) models [22] for liquid time-constant networks [26]. Second, exact linear discretization rules from control theory govern state-space models [6], [24] and their stochastic variants [45]. Third, truncated algebraic path signatures from rough path theory encode continuous trajectories without invoking differential solvers [25]. Despite eliminating solver-induced overhead, these methods introduce specific compromises. As documented in their respective literature, these trade-offs include potential gradient decay in closed-form recurrences [22], constrained expressivity for highly nonlinear dynamics in linear transitions [6], and structural information loss due to signature truncation [25].

While each paradigm offers distinct advantages, none simultaneously satisfies the three requirements identified in Section I. This gap motivates a hybrid design that combines the parallel scalability of fast-weight attention with a continuous-time inductive bias derived from the LTC network—a synthesis absent from prior work and central to our proposed LGA.

### B. Parallelization via Fast Weights and Linear Attention

The self-attention mechanism underpinning Transformers [13] has become a dominant architecture in modern time series modeling [46]. While explicit pairwise interactions afford this mechanism exceptional expressive power, its quadratic complexity in sequence length precludes scaling to the extended horizons typical of complex dynamical systems.

To mitigate this bottleneck, linear attention [47] restructures the attention computation. By replacing the softmax operator with kernel feature maps, this formulation exploits matrix associativity to achieve linear complexity with respect to sequence length. Interpreting the attention state as a matrix-valued memory updated via outer products establishes a formal equivalence with fast-weight programmers [27], [48], providing a principled framework for parallelizing recurrent architectures. Subsequent advances have enriched this framework with data-dependent gating mechanisms, including forget gates in RetNet [49], gated linear attention [50], and the RWKV series [51]–[53]. Concurrently, state-space models such as Mamba [6], [54] and DeltaNet [55] achieve linear complexity through input-dependent transitions and hardware-aware associative scans. Despite their remarkable computational efficiency, these modern variants are predominantly designed for the discrete-time domain [56], where gating operators and state transitions are parameterized by integer step indices.

This historical focus leaves a key open question: can the parallelization principles of fast-weight attention be integrated with continuous-time, input-driven gating dynamics? Addressing this question requires embedding observed temporal distances into fast-weight memory, enabling the architecture to process irregularly sampled time series—a capability absent from existing parallel architectures. This requirement underpins the design of LGA.

## III. Background: Liquid Time-constant Gating

Liquid time-constant networks [26] are continuous-time recurrent neural models. At time $t$, an LTC network determines the $d$-dimensional continuous state vector $\mathbf{s}(t) \in \mathbb{R}^d$ via the following initial value problem (IVP):

$$\frac{\mathrm{d}\mathbf{s}(t)}{\mathrm{d}t} = -\Big[\mathbf{w}_\tau + f\big(\mathbf{s}(t), \mathbf{x}(t)\big)\Big] \odot \mathbf{s}(t) + \mathbf{b}_\tau \odot f\big(\mathbf{s}(t), \mathbf{x}(t)\big), \tag{1}$$

where $\mathbf{x}(t) \in \mathbb{R}^m$ denotes the exogenous input to the system, and $\mathbf{w}_\tau, \mathbf{b}_\tau \in \mathbb{R}^d$ represent the intrinsic inverse time-constant weight and bias, respectively. The function $f(\cdot)$ is a strictly positive, bounded, continuous, and monotonically increasing nonlinearity, while $\odot$ denotes the element-wise product.

*Discussion.* The "liquid" property stems from the input-driven modulation of the system's effective inverse time constants, defined as $\mathbf{w}_\tau + f\big(\mathbf{s}(t), \mathbf{x}(t)\big)$. This mechanism allows individual units to adjust their intrinsic time constants based on the incoming signal, enabling the network to adaptively characterize varying temporal dynamics.

To achieve analytical tractability, we first introduce the one-dimensional LTC configuration which isolates a single scalar state unit $s(t) \in \mathbb{R}$ by removing self-connections [22]. Driven

by a scalar exogenous input $x(t) \in \mathbb{R}$, this continuous state follows a linear non-autonomous variant of Eq. (1):

$$\frac{\mathrm{d}s(t)}{\mathrm{d}t} = -\big[w_\tau + f\big(x(t)\big)\big]s(t) + b_\tau f\big(x(t)\big), \tag{2}$$

where $w_\tau, b_\tau \in \mathbb{R}$ are scalar parameters. To derive the solvable form, we follow the structural symmetry assumption established in [22], which introduces a balancing inverse time-constant parameter into the bias compartment of Eq. (2):

$$\frac{\mathrm{d}s(t)}{\mathrm{d}t} = -\big[w_\tau + f(x(t))\big]s(t) + \big[w_\tau + f(x(t))\big]b_\tau. \tag{3}$$

Under this symmetric condition, applying the linear ODE theory [57] yields the integral solution:

$$s(t) = \big(s(0) - b_\tau\big)\exp\left[-w_\tau t - \int_0^t f\big(x(u)\big)\,\mathrm{d}u\right] + b_\tau, \tag{4}$$

where $s(0) \in \mathbb{R}$ denotes the initial state boundary at $t = 0$. Section S1 in the Supplementary Material details the algebraic verification under this symmetric condition.

Let $t_{n-1}$ and $t_n$ denote two consecutive, irregularly spaced observation timestamps where $t_{n-1} < t_n$. Defining the local temporal interval as $\delta_n = t_n - t_{n-1}$, evaluating the continuous integral solution in Eq. (4) across this boundary yields the following state transition:

$$\frac{s(t_n) - b_\tau}{s(t_{n-1}) - b_\tau} = \exp\left[-w_\tau \delta_n - \int_{t_{n-1}}^{t_n} f\big(x(u)\big)\,\mathrm{d}u\right]. \tag{5}$$

Algebraic rearrangement of Eq. (5) produces a recurrent state update rule modulated by a continuous-time, input-driven gating mechanism:

$$s(t_n) = g(t_n)s(t_{n-1}) + \big[1 - g(t_n)\big]b_\tau, \tag{6}$$

$$g(t_n) = \exp\left[-w_\tau \delta_n - \int_{t_{n-1}}^{t_n} f\big(x(u)\big)\,\mathrm{d}u\right]. \tag{7}$$

*Discussion.* We refer to Eq. (7) as the liquid gate which structurally encapsulates two mathematical properties:

- **Temporal Decay.** The component $-w_\tau \delta_n$ characterizes the intrinsic continuous decay of the state trajectory over the observed temporal distance $\delta_n$.
- **Input Adaptability.** The integral $-\int_{t_{n-1}}^{t_n} f(x(u))\mathrm{d}u$ aggregates the cumulative modulation of exogenous inputs across the same localized time horizon.

The exponential operator maps these properties into a unified and bounded gate $g(t_n) \in (0, 1)$. Derived from LTC dynamics, this formulation endows the liquid gate with continuous-time inductive bias and noise-attenuation properties. This closed-form relation provides the continuous-time, input-driven gating mechanism that underpins LGA Step (1).

## IV. Methodology: Liquid Gated Attention

In this section, we formulate Liquid Gated Attention, a solver-free parallel operator for continuous-time representation learning. Building upon the continuous-time inductive bias established in Section III, the subsequent derivation proceeds through three conceptual layers: (1) a learnable endpoint interpolation that approximates the liquid gate while circumventing numerical integration; (2) an expansion of the scalar state to a matrix-valued associative memory, enabling expressive sequence modeling; and (3) a sequence-level normalization that stabilizes optimization over extended sequences. Consequently, LGA functions as a continuous-time-inspired parallel operator rather than a numerically exact solver translating from the original LTC dynamics.

### A. Deriving the Recurrent Form of LGA

The RNN-like hidden state evolution in the 1D LTC system exposes two limitations: the computational overhead of the integral-based liquid gate and the limited capacity of the scalar hidden state. The following subsections elaborate on the solutions tailored to these two issues, respectively.

*1) A Learnable Integral Approximation:* We approximate the integral-based liquid gate in Eq. (7) via a learnable endpoint interpolation inspired by the trapezoidal rule. Given two consecutive input representations $\mathbf{x}_{n-1}, \mathbf{x}_n \in \mathbb{R}^{1\times d}$, a shared nonlinearity first projects each token into a scalar variable:

$$\begin{aligned} \tilde{x}_{n-1} &= \sigma\left(\mathbf{x}_{n-1}\mathbf{w}_f + b_f\right), \\ \tilde{x}_n &= \sigma\left(\mathbf{x}_n\mathbf{w}_f + b_f\right), \end{aligned} \tag{8}$$

where $\mathbf{w}_f \in \mathbb{R}^{d\times 1}$ and $b_f \in \mathbb{R}$ are learnable parameters, and $\sigma(\cdot)$ denotes the sigmoid activation function.

We then approximate the integral via a learnable weighted average across the boundary endpoints:

$$\int_{t_{n-1}}^{t_n} \tilde{x}(u)\,\mathrm{d}u \approx \delta_n \underbrace{\left[\mu\tilde{x}_{n-1} + (1-\mu)\tilde{x}_n\right]}_{\text{defined as } \overline{x}_n} = \delta_n \overline{x}_n, \tag{9}$$

where $\delta_n = t_n - t_{n-1}$ denotes the time interval. We parameterize the interpolation coefficient as $\mu = \sigma(\theta) \in (0, 1)$ with learnable $\theta \in \mathbb{R}$ to balance each endpoint's contribution. When $\mu = 0.5$, this interpolation recovers the classical trapezoidal rule with second-order local accuracy. In the general learnable case, it serves as a trainable numerical surrogate rather than a fixed quadrature rule. Section S2 of the Supplementary Material provides the proof.

For implementation simplicity, we merge the intrinsic inverse time-constant $w_\tau$ and the input-dependent term into a single decay surrogate, simplifying Eq. (7) to the following efficient liquid gate:

$$g_n = \exp\left[-\delta_n \overline{x}_n\right]. \tag{10}$$

This formulation preserves the temporal decay and input adaptability functionalities of LTC while eliminating the need for numerical integration. This completes LGA Step (2) Computational Efficiency.

*2) An Expressive Hidden State:* We elevate the hidden state from a scalar to a matrix $\mathbf{S}_n \in \mathbb{R}^{d\times d}$ that expands the memory storage capacity for long-range modeling. Furthermore, a dynamic, input-dependent associative memory term substitutes the static bias $b_\tau$ in Eq. (6), improving model flexibility in tracking instantaneous feature variations. Following the linear attention framework [47], the input vector $\mathbf{x}_n$ is projected into key and value vectors $\mathbf{k}_n, \mathbf{v}_n \in \mathbb{R}^{1\times d}$:

$$\begin{aligned} \mathbf{k}_n &= \mathbf{x}_n\mathbf{W}_k/\sqrt{d} + \mathbf{b}_k, \\ \mathbf{v}_n &= \mathbf{x}_n\mathbf{W}_v + \mathbf{b}_v, \end{aligned} \tag{11}$$

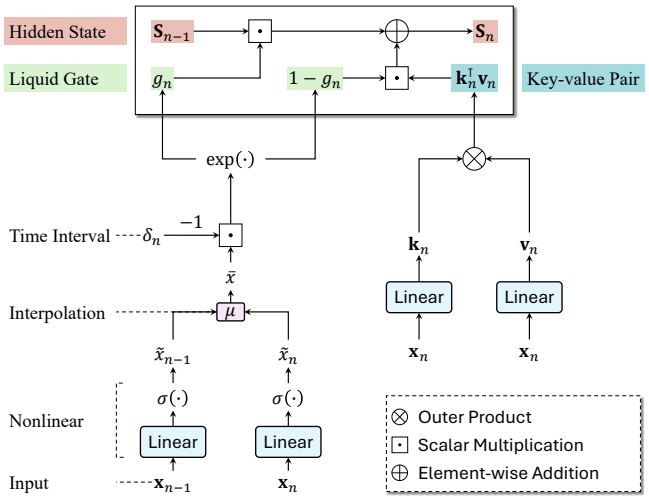



Fig. 1: LGA hidden state update mechanics. The **upper block** depicts the recurrent state transition where the liquid gates modulate the combination of historical memory and associative update matrix. The **lower pipeline** details the liquid gate evaluation workflow incorporating nonlinear projection, learnable interpolation, temporal scaling, and exponential mapping.

where $\mathbf{W}_k, \mathbf{W}_v \in \mathbb{R}^{d\times d}$ and $\mathbf{b}_k, \mathbf{b}_v \in \mathbb{R}^{1\times d}$ are learnable parameters, and $\sqrt{d}$ represents the scaling factor. As visualized in Fig. 1, the recurrent hidden state update rule in LGA is:

$$\mathbf{S}_n = g_n \mathbf{S}_{n-1} + (1-g_n)\mathbf{k}_n^\top \mathbf{v}_n. \tag{12}$$

*Discussion.* Eq. (12) implements the fast-weight programming scheme [27], where $\mathbf{S}_n$ acts as an associative memory updated via vector outer products. This elevation expands the representational capacity into a $d\times d$ interaction matrix [52], [54], [58], fulfilling LGA Step (3) Expressive Capability.

The output retrieval in LGA handles the feature extraction and state readout from associative memory matrices. At time $t_n$, the input representation $\mathbf{x}_n$ is mapped to the query vector $\mathbf{q}_n \in \mathbb{R}^{1\times d}$ and the output gate $\mathbf{o}_n \in \mathbb{R}^{1\times d}$ via:

$$\mathbf{q}_n = \mathbf{x}_n \mathbf{W}_q + \mathbf{b}_q, \tag{13}$$

$$\mathbf{o}_n = \sigma\left(\mathbf{x}_n \mathbf{W}_o + \mathbf{b}_o\right), \tag{14}$$

where $\mathbf{W}_q, \mathbf{W}_o \in \mathbb{R}^{d\times d}$ and $\mathbf{b}_q, \mathbf{b}_o \in \mathbb{R}^{1\times d}$ are learnable weights and biases, respectively. The output gate then modulates the retrieved context within the hidden state matrix $\mathbf{S}_n$ to produce the final output vector $\mathbf{y}_n \in \mathbb{R}^{1\times d}$ via:

$$\mathbf{y}_n = \mathbf{o}_n \odot \left(\mathbf{q}_n \mathbf{S}_n\right). \tag{15}$$

The output gate allows the model to suppress irrelevant or noisy retrieved channels, improving robustness against high-frequency observational perturbations.

### *B. Deriving the Parallel Former of LGA*

Assuming a zero-initialized hidden state $\mathbf{S}_0 = \mathbf{0}$, unrolling the recurrent update rule in Eq. (12) across the temporal horizon yields:

$$\mathbf{S}_n = \sum_{i=1}^{n} \left[ \left( \prod_{j=i+1}^{n} g_j \right) (1-g_i)\, \mathbf{k}_i \right]^\top \mathbf{v}_i. \tag{16}$$

Substituting Eq. (16) into the output retrieval expression in Eq. (15) and factoring out the $n$-dependent terms from the temporal summation yields:

$$\mathbf{y}_n = \mathbf{o}_n \odot \left[ (\dot{g}_n \mathbf{q}_n) \sum_{i=1}^{n} \left[ (1-g_i)\, \mathbf{k}_i \right]^\top \left( \frac{\mathbf{v}_i}{\dot{g}_i} \right) \right], \tag{17}$$

where $\dot{g}_n = \prod_{i=1}^{n} g_i \in \mathbb{R}$ represents the cumulative liquid gate product. Under zero initialization, Eq. (17) remains algebraically equivalent to the recurrent updates in Eq. (12). Temporal index alignment enables these liquid gates to directly scale token representations, transforming the continuous-time inductive bias into explicit modulation coefficients $g_n$ and $\dot{g}_n$ for the query, key, and value vectors.

Building upon Eq. (17), exploiting matrix associativity within the fast-weight programming framework [27] can eliminate sequential dependencies in the recurrent state. This yields two linearly scaling parallel modes for causal and non-causal computation, depending on the temporal aggregation boundaries. Given the modulated features $\dot{\mathbf{Q}}, \dot{\mathbf{K}}, \dot{\mathbf{V}} \in \mathbb{R}^{n\times d}$ and the element-wise output gate $\mathbf{O} \in \mathbb{R}^{n\times d}$, the parallel matrix transformations in LGA that map these inputs to the output matrix $\mathbf{Y} \in \mathbb{R}^{n\times d}$ are instantiated as:

- **Causal Mode.** Online streaming and autoregressive generation tasks strictly prohibit the future information leakage. Leveraging a parallel prefix scan, the causal LGA operator is formulated as:

$$\mathbf{Y} = \mathbf{O} \odot \left[ \dot{\mathbf{Q}} \circledast \mathrm{CumSum}\left( \dot{\mathbf{K}} \otimes \dot{\mathbf{V}} \right) \right], \tag{18}$$

  where $\otimes$ and $\circledast$ denote the row-wise outer product and vector-matrix multiplication, respectively. $\mathrm{CumSum}(\cdot)$ computes the associative cumulative sum along the temporal dimension. This implementation maintains causal semantics while reducing both computational and memory complexity to $\mathcal{O}(nd^2)$.
- **Non-causal Mode.** For offline representation learning tasks where the strict causality is unnecessary, the temporal aggregation directly encompasses the full sequence. Replacing the prefix scan with a global temporal summation, LGA functions as a bidirectional encoder that

$$\mathbf{Y} = \mathbf{O} \odot \left[ \dot{\mathbf{Q}} \left( \dot{\mathbf{K}}^\top \dot{\mathbf{V}} \right) \right], \tag{19}$$

  which prioritizes the matrix product between the key and value matrices to preserve the linear complexity $\mathcal{O}\left(nd^2\right)$.

We summarize Eq. (18) and Eq. (19) into a unified form:

$$\mathbf{Y} = \mathrm{LGA}(\mathbf{X}, \boldsymbol{\Delta}), \tag{20}$$

since liquid gates, modulated features and output gate are calculated based on the input sequence representation matrix $\mathbf{X} \in \mathbb{R}^{n\times d}$ and the corresponding time interval vector $\boldsymbol{\Delta} \in \mathbb{R}^{n\times 1}$. Figure 2 visualizes the LGA pipeline.

The base features $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{n\times d}$ and the output gate $\mathbf{O}$ are obtained via linear projections of the input matrix $\mathbf{X}$:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_q + \mathbf{b}_q, \tag{21}$$

$$\mathbf{K} = \mathbf{X}\mathbf{W}_k/\sqrt{d} + \mathbf{b}_k, \tag{22}$$

$$\mathbf{V} = \mathbf{X}\mathbf{W}_v + \mathbf{b}_v, \tag{23}$$

$$\mathbf{O} = \sigma\left(\mathbf{X}\mathbf{W}_o + \mathbf{b}_o\right). \tag{24}$$

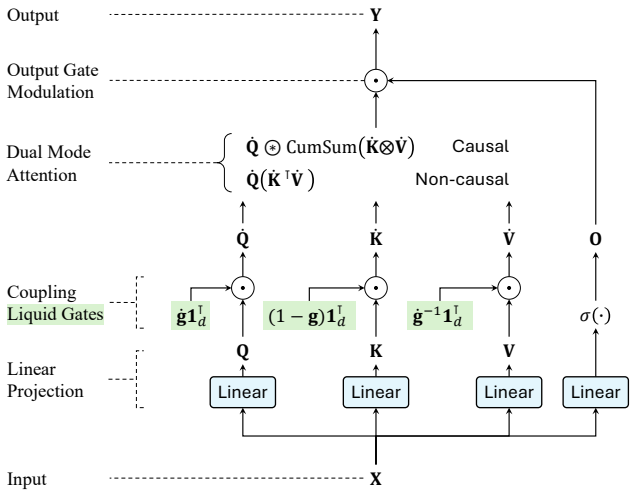



Fig. 2: Parallel LGA pipeline. The input representation $\mathbf{X}$ is first projected and scaled by vectorized liquid gates to embed continuous-time gating dynamics, yielding modulated representations. The dual mode attention block then executes either a causal prefix scan or a global matrix multiplication with linear complexity. The output gate modulates the aggregated state to produce the final representation. $\odot$, $\circledast$, and $\otimes$ denote the element-wise product, row-wise vector-matrix multiplication, and row-wise matrix outer product, respectively. $\mathrm{CumSum}(\cdot)$ computes the cumulative sum along the temporal dimension.

The modulated features $\dot{\mathbf{Q}}, \dot{\mathbf{K}}, \dot{\mathbf{V}} \in \mathbb{R}^{n\times d}$ are obtained by scaling with the vectorized liquid gates $\mathbf{g}, \dot{\mathbf{g}} \in \mathbb{R}^{n\times 1}$:

$$\dot{\mathbf{Q}} = \mathbf{Q} \odot \left(\dot{\mathbf{g}}\mathbf{1}_d^\top\right), \tag{25}$$

$$\dot{\mathbf{K}} = \mathbf{K} \odot \left[(1-\mathbf{g})\,\mathbf{1}_d^\top\right], \tag{26}$$

$$\dot{\mathbf{V}} = \mathbf{V} \odot \left(\dot{\mathbf{g}}^{-1}\mathbf{1}_d^\top\right). \tag{27}$$

where the scalar 1 performs element-wise broadcasting and the vector $\mathbf{1}_d^\top \in \mathbb{R}^{1\times d}$ denotes the channel-wise broadcasting.

The parallel computation of the liquid gate vector $\mathbf{g}$ is performed by applying the operations in Eqs. (8)—(10) simultaneously across the temporal dimension, as formalized in Algorithm 1. To evaluate the cumulative liquid gate $\dot{\mathbf{g}}$, the sequential multiplication chain is mapped into log-space which reformulates the recurrent dependency into the cumulative summation, yielding

$$\dot{\mathbf{g}} = \exp\left[-\mathrm{CumSum}\left(\boldsymbol{\Delta} \odot \overline{\mathbf{x}}\right)\right] \tag{28}$$

where $\overline{\mathbf{x}} \in \mathbb{R}^{n\times 1}$ represents the interpolated nonlinear vector.

*1) Sequence-level Normalization:* Due to the cumulative term $\dot{\mathbf{g}}$, the vanilla parallel execution is prone to numerical instability over long sequences. Since the inputs are non-negative, the unconstrained prefix sum in Eq. (28) grows monotonically with the sequence length, leading to exponential underflow and vanishing gradients.

To stabilize long-horizon optimization, we introduce a sequence-level normalization mechanism. Let $\mathbf{u} = \boldsymbol{\Delta} \odot \overline{\mathbf{x}} \in \mathbb{R}^{n\times 1}$. The scaling vector $\mathbf{u}$ is normalized by its summation along the sequence dimension:

$$\hat{\mathbf{u}} = \frac{\mathbf{u}}{\sum_{i=1}^{n} u_i + \epsilon}, \tag{29}$$

where $u_i = \delta_i \overline{x}_i$ is the $i$-th element of $\mathbf{u}$, and $\epsilon$ is a small constant added to prevent division by zero. Substituting $\mathbf{u}$ with $\hat{\mathbf{u}}$ bounds the cumulative sum within $(0, 1]$. Consequently, the exponential scaling coefficients remain constrained to $(e^{-1}, 1]$, as proven in Section S3 of the Supplementary Material. Thus, the stable local and cumulative liquid gates are formulated as:

$$\mathbf{g}_{\text{stable}} = \exp\left(-\hat{\mathbf{u}}\right), \tag{30}$$

$$\dot{\mathbf{g}}_{\text{stable}} = \exp\left[-\mathrm{CumSum}\left(\hat{\mathbf{u}}\right)\right]. \tag{31}$$

Algorithm 1 formalizes the complete parallel evaluation flow of these liquid gate vectors, including the alternative numerical stabilization path.

*Discussion.* The normalized gate is not algebraically equivalent to the original LTC state transition. Instead, it operates as a sequence-level regularized surrogate that preserves relative temporal distance allocation while bounding the total decay budget. Moreover, this mechanism reduces the gradient decay from exponential to polynomial as the sequence length grows (see Section S3 of the Supplementary Material). This enables stable optimization over extended sequences, fulfilling Step (4) Numerical Safety. This design targets offline full-sequence representation learning; online streaming or causal inference requires a prefix-normalized variant to eliminate dependence on future observations.

*2) A Decoupled Multi-Head Extension:* To scale the continuous-time gating dynamics to high-dimensional spaces, we derive a decoupled multi-head LGA mechanism.

Specifically, the $d$-dimensional latent space is partitioned into $H$ distinct subspaces, where each head $h \in \{1, \ldots, H\}$

**Algorithm 1** Parallel Computation of Liquid Gates under Vanilla and Stabilized Modes

**Input:** Input matrix $\mathbf{X} \in \mathbb{R}^{n\times d}$, time interval vector $\boldsymbol{\Delta} \in \mathbb{R}^{n\times 1}$, weight vector $\mathbf{w}_f \in \mathbb{R}^{d\times 1}$, bias scalar $b_f \in \mathbb{R}$, unconstrained parameter $\theta \in \mathbb{R}$, stability constant $\epsilon > 0$, mode selector $\mathcal{M} \in \{\text{Vanilla}, \text{Stabilized}\}$.

**Output:** Local liquid gate vector $\mathbf{g} \in \mathbb{R}^{n\times 1}$ and cumulative liquid gate vector $\dot{\mathbf{g}} \in \mathbb{R}^{n\times 1}$.

$\mu \leftarrow \sigma(\theta)$ ▷ Bounded interpolation coefficient
$\tilde{\mathbf{x}} \leftarrow \sigma(\mathbf{X}\mathbf{w}_f + b_f)$ ▷ Nonlinear projection
$\tilde{\mathbf{x}}_{\text{prev}} \leftarrow [0; \tilde{x}_1; \ldots; \tilde{x}_{n-1}]$ ▷ Temporal shift
$\overline{\mathbf{x}} \leftarrow \mu\tilde{\mathbf{x}}_{\text{prev}} + (1-\mu)\tilde{\mathbf{x}}$ ▷ Interpolation
$\mathbf{u} \leftarrow \boldsymbol{\Delta} \odot \overline{\mathbf{x}}$ ▷ Base decay vector
**if** $\mathcal{M} = \text{Stabilized}$ **then**
  $\hat{\mathbf{u}} \leftarrow \mathbf{u} \,/\, (\sum_{i=1}^{n} u_i + \epsilon)$ ▷ Sequence-level normalization
  $\mathbf{g} \leftarrow \exp(-\hat{\mathbf{u}})$ ▷ Stabilized local gate
  $\dot{\mathbf{g}} \leftarrow \exp(-\mathrm{CumSum}(\hat{\mathbf{u}}))$ ▷ Stabilized cumulative gate
**else**
  $\mathbf{g} \leftarrow \exp(-\mathbf{u})$ ▷ Vanilla local gate
  $\dot{\mathbf{g}} \leftarrow \exp(-\mathrm{CumSum}(\mathbf{u}))$ ▷ Vanilla cumulative gate
**end if**
**return** $\mathbf{g}, \dot{\mathbf{g}}$

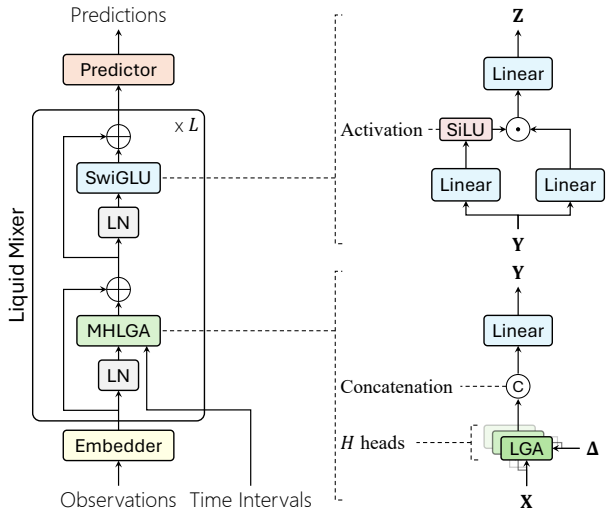



Fig. 3: LFormer architecture. The **left block** depicts the macro design, where $L$ stacked Liquid Mixers connect a task-specific Embedder and Predictor. The **right panels** detail the MHLGA (bottom) and SwiGLU (upper) workflows.

operates on a subspace of dimension $d_h = d/H$. Let $\mathbf{W}^{(h)}$, $\mathbf{b}^{(h)}$, and $\mu^{(h)}$ denote the head-specific learnable projection weights, biases, and adaptive interpolation coefficients for the $h$-th subspace, respectively. Each partitioned head executes the linear projection, adaptive interpolation, and sequence-level normalization procedures independently, yielding the decoupled subspace representations:

$$\mathbf{Y}^{(h)} = \mathrm{LGA}^{(h)}\left(\mathbf{X}, \mathbf{\Delta}\right), \quad h \in \{1, \ldots, H\}. \tag{32}$$

Subsequently, the outputs from all $H$ heads are concatenated along the feature dimension and linearly projected to construct the final multi-head representation:

$$\mathbf{Y} = \mathrm{MHLGA}\left(\mathbf{X}, \mathbf{\Delta}\right) = \mathrm{Concat}\left(\mathbf{Y}^{(1)}, \ldots, \mathbf{Y}^{(H)}\right)\mathbf{W}_m, \tag{33}$$

where $\mathrm{Concat}\left(\cdot\right)$ denotes the concatenation operator and $\mathbf{W}_m \in \mathbb{R}^{d\times d}$ represents the terminal projection weight matrix.

*Discussion.* Unlike standard multi-head attention that primarily captures heterogeneous subspace features [13], the key architectural advancement lies in the head-wise decoupling of continuous-time gating dynamics. Equipping each head with independent interpolation parameters and liquid gates enhances the model's expressive capability at the system level, enabling collaborative modeling of complex temporal patterns.

### C. LFormer: An Instantiation of LGA

Building upon the LGA operator, we establish a modular backbone for continuous-time representation learning system, designated as LFormer. As illustrated in Fig. 3, the architecture employs a three-stage processing pipeline consisting of a task-specific Embedder, $L$ stacked Liquid Mixers, and a task-oriented Predictor.

*1) Embedder:* The Embedder projects raw input observations into a $d$-dimensional representation space. To accommodate variable-length sequences, we define a canonical sequence length $n$. The longer sequences are partitioned into non-overlapping sub-sequences of length $n$, while shorter sequences are extended to length $n$ via a zero-padding scheme. The padding tokens are excluded from the gradient optimization. Formally, given a raw or pre-processed time series with $m$ features $\mathbf{X}_{\text{raw}} \in \mathbb{R}^{n\times m}$, the Embedder operates via:

$$\mathbf{X} = \mathrm{Embed}\left(\mathbf{X}_{\text{raw}}\right), \tag{34}$$

where $\mathrm{Embed}\left(\cdot\right) : \mathbb{R}^m \to \mathbb{R}^d$ denotes a task-specific transformation mapping raw observations to the latent sequence representation matrix $\mathbf{X} \in \mathbb{R}^{n\times d}$.

*2) Liquid Mixer:* The Liquid Mixer serves as the core block of LFormer, designed to refine latent representations through interleaved temporal and channel-wise transformations. The framework stacks $L$ identical mixer layers, where the $l$-th mixer ($l \in \{1, \ldots, L\}$) processes the representations generated by its predecessor or the initialization from the Embedder.

Specifically, each mixer layer consists of a multi-head liquid gated attention (MHLGA) module and a swish-gated linear unit (SwiGLU), organized in a residual connection structure with pre-layer normalization (LN):

$$\mathbf{Y} = \mathbf{X} + \mathrm{MHLGA}\left(\mathrm{LN}\left(\mathbf{X}\right), \mathbf{\Delta}\right), \tag{35}$$

$$\mathbf{Z} = \mathbf{Y} + \mathrm{SwiGLU}\left(\mathrm{LN}\left(\mathbf{Y}\right)\right), \tag{36}$$

where $\mathbf{Z} \in \mathbb{R}^{n\times d}$ denotes the output of the current mixer layer. The MHLGA component operates along the temporal sequence dimension to inject continuous-time inductive bias via decoupled liquid gates, whereas the SwiGLU block modulates cross-feature channel interactions to enhance non-linear expressivity. For any hidden representation $\mathbf{H} \in \mathbb{R}^{n\times d}$, the operation in SwiGLU is:

$$\mathrm{SwiGLU}\left(\mathbf{H}\right) = \left[\mathrm{SiLU}\left(\mathbf{H}\mathbf{W}_1\right) \odot \left(\mathbf{H}\mathbf{W}_2\right)\right]\mathbf{W}_3, \tag{37}$$

where $\mathbf{W}_1, \mathbf{W}_2 \in \mathbb{R}^{d\times d_{ff}}$ and $\mathbf{W}_3 \in \mathbb{R}^{d_{ff}\times d}$ represent learnable weight matrices, and $\mathrm{SiLU}\left(x\right) = x\cdot\sigma\left(x\right)$ represents the sigmoid linear unit.

*3) Predictor:* The final output of the terminal Liquid Mixer layer, denoted as $\mathbf{Z}^{(L)} \in \mathbb{R}^{n\times d}$, serves as the high-level semantic representation of the input sequence. The Predictor maps this representation into the target task space:

$$\mathbf{P} = \mathrm{Pred}(\mathbf{Z}^{(L)}), \tag{38}$$

where $\mathbf{P}$ represents the task-oriented target output (e.g., forecasted future trajectories or classification probabilities). Depending on downstream constraints, the operator $\mathrm{Pred}\left(\cdot\right)$ can be instantiated as a linear projection head, a global pooling operator, or a multilayer perceptron (MLP).

## V. Experiments and Discussion

This section empirically evaluates LFormer around three core research questions (RQs):

- **RQ1 (Comparative Performance):** How does LFormer perform against representative state-of-the-art discrete-time and continuous-time baselines across diverse temporal tasks, regarding long-range dependency modeling, fine-grained state tracking, and trajectory reconstruction from sparse and noisy observations?

TABLE I: A Summary of 16 Datasets across 6 Tasks

| Task | Dataset | Irregular | Length | Dimension | Target | Size |
|---|---|---|---|---|---|---|
| LTS-C | EC | No | 1,751 | 3 | 4 | 524 |
| | EW | | 17,984 | 6 | 5 | 236 |
| | HB | | 405 | 61 | 2 | 409 |
| | MI | | 3,000 | 64 | 2 | 378 |
| | SCP1 | | 896 | 6 | 2 | 561 |
| | SCP2 | | 1,152 | 7 | 2 | 380 |
| LTS-R | HR | No | 4,000 | 2 | 1 | 7,949 |
| | RR | | | | | 7,870 |
| | $SpO_2$ | | | | | 7,949 |
| PTS-C | HA | Yes | 50 | 12 | 7 | 6,554 |
| PTS-R | PDL | Yes | 50 | $24 \times 24$ | 2 | 4,000 |
| TS-I TS-E | $2DS_\alpha$ | Yes | 150 | 2 | 2 | 300 |
| | USH | | 730 | 5 | 5 | 1,192 |
| | PHY | | 72.16 | 37 | 37 | 8,000 |

Note: **Irregular** specifies the presence of irregular sampling; **Dimension** and **Size** denote the feature count and total instances, while **Target** indicates the number of classes or continuous values to be predicted. $2DS_\alpha$ refers to the 2D Spiral dataset variants with $\alpha \in \{0.02, 0.2, 2\}$. Due to the variable-length nature of PHY, its reported length reflects the sample mean.

- **RQ2 (Ablation Analysis):** How do the individual components of LFormer, including the continuous-time liquid gate dynamics, sequence-level normalization, and decoupled multi-head mechanism, contribute to its representational capacity and numerical stability?
- **RQ3 (Computational Efficiency):** Does LFormer achieve linear computational and memory complexity under both non-causal and causal settings while maintaining competitive accuracy over extended sequence horizons?

### A. Experimental Setup

*1) Tasks and Datasets :* We adopt a benchmark across six temporal modeling tasks spanning sixteen datasets to evaluate the three core capabilities defined in RQ1:

- **Long-term Time Series Classification and Regression (LTS-C and LTS-R)** evaluate the capacity to model long-range dependencies by mapping global trajectories to categorical labels or continuous scalars. The classification setup incorporates six datasets {EC, EW, HB, MI, SCP1, SCP2} from the UEA[1] archive [59] whereas the regression configuration utilizes three datasets {HR, RR, $SpO_2$} from the BIDMC[2] database [60] to infer patient vital signs from dense, 4,000-step physiological signals.
- **Per-point Time Series Classification and Regression (PTS-C and PTS-R)** assess the capability for fine-grained state tracking by decoding latent states at arbitrary, irregularly sampled timestamps. For point-wise classification, the Human Activity (HA) dataset[3] [5] is employed to classify localized actions at each time point, using 50 irregular observations per sequence. For point-wise regression, the synthetic Pendulum (PDL) dataset [61] is adopted to infer underlying kinematic angles from noisy image sequences spanning 50 irregularly sampled frames [9].
- **Time Series Interpolation and Extrapolation (TS-I and TS-E)** evaluate trajectory reconstruction from sparse and noisy observations by recovering continuous-time dynamics under partial observation constraints. We adopt three datasets. The synthetic 2D spiral datasets ($2DS_\alpha$) [10] generate 150-step spiral trajectories with shape variance controlled by $\alpha \in \{2, 0.2, 0.02\}$; models observe 30 randomly sampled points from the first half and reconstruct the full trajectory. The USHCN (USH) dataset[4] [62] comprises daily climate measurement (5 variables) from 1,192 weather stations across the United States over a four-year period. Following [9], we subsample 50% of the time points and randomly drop 20% of the remaining observations to obtain irregularly sampled sequences. The Physionet (PHY) dataset[5] [63] contains 8,000 multivariate clinical time series from intensive care unit patients, each recording 37 time-varying features over 48 hours. For both USH and PHY, we evaluate preprocessing, interpolation and extrapolation following the protocols in [45].

Table I summarizes the structural characteristics of these datasets. Section S4 of the Supplementary Material provides comprehensive preprocessing details, including irregular sampling protocols and train/validation/test partitioning schemas.

*2) Baselines :* We benchmark LFormer against state-of-the-art baselines from three paradigms:

- **Discrete-time Models.** This paradigm encompasses architectures engineered for uniformly indexed sequences without explicit physical time tracking. Representatives include GRU [11] and Transformer [13]. Modern linear recurrent and discrete state-space models are incorporated, including LRU [64] and Mamba (S6) [6]. We also include baselines that augment discrete-time models with time-interval inputs or decay mechanisms: GRU-$\Delta_t$ [65], GRU-D [32], and RKN-$\Delta_t$ [61].
- **Solver-dependent Continuous-time Models.** This paradigm comprises architectures that parameterize latent state derivatives with neural networks and evaluate hidden trajectories via numerical differential solvers. This setup features vector-field equations ( NODE [19], R-ODE [19], O-ODE [5], LSDE [66]), continuous path integrals (NCDE [20], NRDE [21], LogNCDE [67]), solver-driven continuous-time attention ContiFormer [10], stochastic filtering CRU [9] and Bayesian extension GRU-ODE-B [68].
- **Solver-free Continuous-time Models:** This paradigm groups architectures that evaluate trajectory representations analytically or via exact linear discretization rules, bypassing iterative integration overhead. This setup incorporates closed-form liquid dynamics CfC [22], truncated algebraic path signatures RFormer [25], multi-time embedding kernels (mTAND [69], and time-varying continuous state-space formulations (S5 [70], ACSSM [45]).

Detailed description of all baselines is provided in Section S5 of the Supplementary Material.

[1]https://timeseriesclassification.com
[2]https://physionet.org/content/bidmc/1.0.0/
[3]https://doi.org/10.24432/C57G8X
[4]https://data.ess-dive.lbl.gov/view/doi:10.3334/CDIAC/CLI.NDP019
[5]https://physionet.org/content/challenge-2012/1.0.0/

TABLE II: Long-term Time Series Classification Accuracy (%, mean$_{\pm \text{std}}$) on Six UEA Datasets

| Dataset | Method | | | | | | | | | | $\Delta_\uparrow$ |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | LRU† [64] | S5† [70] | S6† [6] | Mamba† [6] | NCDE† [20] | NRDE† [21] | LogNCDE† [67] | Transformer† [13] | RFormer† [25] | LFormer (ours) | |
| EC | 21.5$_{\pm 2.1}$ | 24.1$_{\pm 4.3}$ | 26.4$_{\pm 6.4}$ | 27.9$_{\pm 4.5}$ | 29.9$_{\pm 6.5}$ | 25.3$_{\pm 1.8}$ | 34.4$_{\pm 6.4}$ | 40.5$_{\pm 6.3}$ | 34.7$_{\pm 4.1}$ | **45.3**$_{\pm 3.5}$ | +11.9% |
| EW | 87.8$_{\pm 2.8}$ | 81.1$_{\pm 3.7}$ | 85.0$_{\pm 16.1}$ | 70.9$_{\pm 15.8}$ | 75.0$_{\pm 3.9}$ | 83.9$_{\pm 7.3}$ | 85.6$_{\pm 5.1}$ | OOM | **90.3**$_{\pm 0.1}$ | 89.4$_{\pm 3.2}$ | -1.0% |
| HB | 78.4$_{\pm 6.7}$ | 77.7$_{\pm 5.5}$ | 76.5$_{\pm 8.3}$ | 76.2$_{\pm 3.8}$ | 73.9$_{\pm 2.6}$ | 72.9$_{\pm 4.8}$ | 75.2$_{\pm 4.6}$ | 70.5$_{\pm 0.1}$ | 72.5$_{\pm 0.1}$ | **81.6**$_{\pm 1.6}$ | +4.1% |
| MI | 48.4$_{\pm 5.0}$ | 47.7$_{\pm 5.5}$ | 51.3$_{\pm 4.7}$ | 47.7$_{\pm 4.5}$ | 49.5$_{\pm 2.8}$ | 47.0$_{\pm 5.7}$ | 53.7$_{\pm 5.3}$ | 50.5$_{\pm 3.0}$ | 55.8$_{\pm 6.6}$ | **65.3**$_{\pm 4.1}$ | +17.0% |
| SCP1 | 82.6$_{\pm 3.4}$ | **89.9**$_{\pm 4.6}$ | 82.8$_{\pm 2.7}$ | 80.7$_{\pm 1.4}$ | 79.8$_{\pm 5.6}$ | 80.9$_{\pm 2.5}$ | 83.1$_{\pm 2.8}$ | 84.3$_{\pm 6.3}$ | 81.2$_{\pm 2.8}$ | 84.5$_{\pm 1.7}$ | -6.0% |
| SCP2 | 51.2$_{\pm 3.6}$ | 50.5$_{\pm 2.6}$ | 49.9$_{\pm 9.5}$ | 48.2$_{\pm 3.9}$ | 53.0$_{\pm 2.8}$ | 53.7$_{\pm 6.9}$ | 53.7$_{\pm 4.1}$ | 49.1$_{\pm 2.5}$ | 52.3$_{\pm 3.7}$ | **61.8**$_{\pm 3.0}$ | +15.1% |
| **Avg. Acc.$_\uparrow$** | 61.7 | 61.8 | 62.0 | 58.6 | 60.2 | 60.6 | 64.3 | 49.2 | 64.5 | **71.3** | +10.5% |
| **Avg. Rank$_\downarrow$** | 5.67 | 5.92 | 5.50 | 7.92 | 6.67 | 7.08 | 3.92 | 6.50 | 4.50 | **1.33** | - |

Note: The best results are highlighted in **bold**, and the second best are underlined. $\Delta$ denotes the relative performance gain of LFormer over the best competitor. "Avg. Acc." denotes the average accuracy over six datasets, while "Avg. Rank" denotes the average ranking across all datasets (fractional ranking is used for ties; OOM is ranked last). The arrows $\uparrow$ and $\downarrow$ indicate whether a higher or lower value is better. OOM indicates Out-Of-Memory. Results marked with † are taken from [25] under identical dataset splits and evaluation metrics.

TABLE III: Long-term Time Series Regression MAE and RMSE (mean$_{\pm \text{std}}$) on Three BIDMC Datasets

| Dataset | Metric | Method | | | | | | | | | $\Delta_\uparrow$ |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | GRU [11] | Mamba [6] | Transformer [13] | R-ODE [5] | NCDE [20] | NRDE [21] | CfC [22] | RFormer [25] | LFormer (ours) | |
| HR | MAE$_\downarrow$ | 9.64$_{\pm 0.61}$ | 1.97$_{\pm 0.39}$ | 9.82$_{\pm 0.19}$ | 12.11$_{\pm 0.00}$ | 9.20$_{\pm 0.08}$ | 2.74$_{\pm 0.13}$ | 9.77$_{\pm 0.31}$ | 2.37$_{\pm 0.17}$ | **1.03**$_{\pm 0.04}$ | +47.72% |
| | RMSE$_\downarrow$ | 13.01$_{\pm 0.69}$ | 2.75$_{\pm 0.70}$ | 13.22$_{\pm 0.16}$ | 15.17$_{\pm 0.00}$ | 11.41$_{\pm 0.12}$ | 3.45$_{\pm 0.22}$ | 13.12$_{\pm 0.39}$ | 3.09$_{\pm 0.23}$ | **1.52**$_{\pm 0.07}$ | +44.73% |
| RR | MAE$_\downarrow$ | 1.79$_{\pm 0.36}$ | 0.93$_{\pm 0.60}$ | 1.45$_{\pm 0.22}$ | 1.85$_{\pm 0.00}$ | 2.27$_{\pm 0.03}$ | 1.20$_{\pm 0.04}$ | 2.15$_{\pm 0.12}$ | 1.06$_{\pm 0.04}$ | **0.74**$_{\pm 0.04}$ | +20.43% |
| | RMSE$_\downarrow$ | 2.51$_{\pm 0.47}$ | 1.32$_{\pm 0.10}$ | 1.94$_{\pm 0.25}$ | 2.32$_{\pm 0.35}$ | 2.84$_{\pm 0.04}$ | 1.51$_{\pm 0.06}$ | 2.98$_{\pm 0.16}$ | 1.46$_{\pm 0.05}$ | **0.97**$_{\pm 0.02}$ | +26.52% |
| $SpO_2$ | MAE$_\downarrow$ | 2.46$_{\pm 0.03}$ | 1.08$_{\pm 0.49}$ | 2.45$_{\pm 0.03}$ | 2.71$_{\pm 0.00}$ | 2.46$_{\pm 0.12}$ | 1.02$_{\pm 0.09}$ | 2.46$_{\pm 0.03}$ | 1.38$_{\pm 0.15}$ | **0.29**$_{\pm 0.03}$ | +71.57% |
| | RMSE$_\downarrow$ | 3.28$_{\pm 0.08}$ | 1.41$_{\pm 0.71}$ | 3.28$_{\pm 0.08}$ | 3.41$_{\pm 0.02}$ | 2.83$_{\pm 0.18}$ | 1.28$_{\pm 0.13}$ | 3.28$_{\pm 0.08}$ | 1.80$_{\pm 0.22}$ | **0.42**$_{\pm 0.02}$ | +67.19% |

Note: The best results are highlighted in **bold**, and the second best are underlined. $\Delta$ denotes the relative performance gain of LFormer over the best competitor. The arrows $\uparrow$ and $\downarrow$ indicate whether a higher or lower value is better.

*3) Evaluation Metrics :* For classification tasks (LTS-C, PTS-C), we report Top-1 Accuracy. For LTS-R, we report Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE); for PTS-R, we report Mean Squared Error (MSE). For TS-I and TS-E, we report MAE and RMSE on $2DS_\alpha$, and MSE on PHY and USH.

*4) Implementation Details :* We adopt a shared base configuration across the majority of benchmarks: $L = 2$ Liquid Mixers, model dimension $d = 64$, SwiGLU expansion $d_{ff} = 176$, and $H = 4$ attention heads. This default setup is used for all LTS-R tasks and four of the six LTS-C datasets (EC, EW, HB, MI). Task-specific adaptations are applied where required by the data modality or objective: SCP1 and SCP2 use $d = 128$, $d_{ff} = 352$, $H = 1$, and $L = 1$; PTS-R employs a convolutional embedder with $d = 128$ and $d_{ff} = 352$ to process $24\times24$ pixel inputs; PTS-C stacks $L = 4$ layers for fine-grained per-point classification; TS-I/E on $2DS_\alpha$ adopts $d = 32$, $L = 1$, and a continuous time embedding following [10]; USH and PHY use an MLP embedder with two hidden layers. Complete per-task specifications are provided in Section S6 of the Supplementary Material.

The optimizer is Adam with a learning rate of $10^{-3}$ (exceptions: $10^{-2}$ for PTS-R, TS-I/E on $2DS_\alpha$). Early stopping is applied when the validation metric does not improve for 20 consecutive epochs.To promote fair comparison, we calibrate baselines via grid search with the identical early stop strategy, or directly report published results when the evaluation protocol exactly matches ours. All experiments are run with five fixed random seeds $\{42, 43, 44, 45, 46\}$ and we report mean and standard deviation. All execution pipelines are deployed on a standardized workstation hardware environment equipped with an Intel Core i9-13900K CPU, 128 GB RAM, and a single NVIDIA RTX 4090 GPU.

## B. Temporal Modeling Capability Evaluation (RQ 1)

We evaluate LFormer against state-of-the-art baselines along three core temporal modeling dimensions as follows.

*1) Long-range Dependency Modeling (LTS-C and LTS-R) :* As summarized in Table II, LFormer achieves the highest average accuracy (71.3%) and the top average rank (1.33) on the LTS-C benchmark, delivering the top performance on four out of six UEA datasets. On EW and SCP1, it ranks second behind RFormer and S5, respectively, indicating that signature-based or linear state-space inductive biases can still offer competitive advantages in certain classification regimes. Notably, LFormer secures a top-two finish across all evaluated datasets, whereas no competing baseline achieves a top-two ranking on more than three. On the extended-horizon EW dataset (17,984 steps), LFormer retains robust optimization stability and computational tractability, whereas the canonical Transformer incurs out-of-memory (OOM) exceptions. This result empirically confirms the scalability benefit of LGA's linear-complexity design.

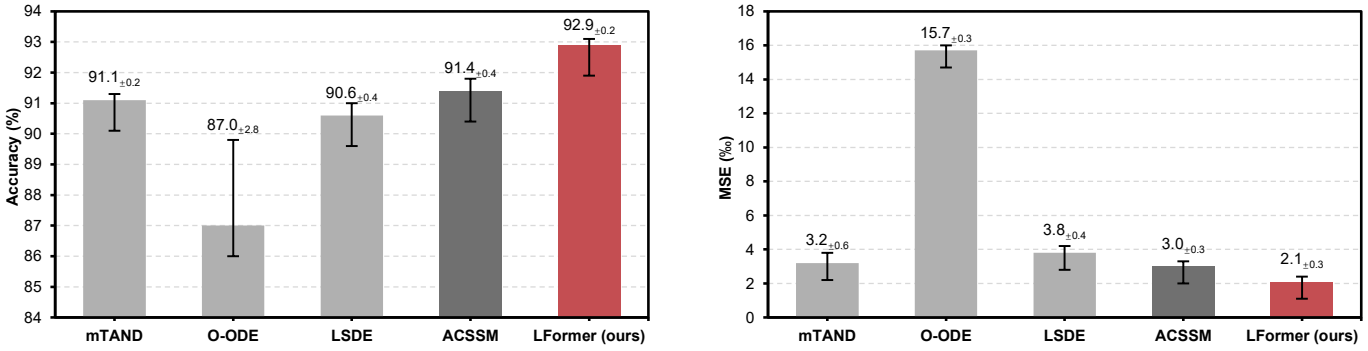
The empirical advantage of LFormer extends to the LTS-R benchmark. As detailed in Table III, LFormer consistently outperforms all baselines across three evaluated BIDMC datasets in terms of both MAE and RMSE. Compared to the strongest baseline, LFormer achieves relative MAE reductions of 47.72% for HR, 20.43% for RR, and 71.57% for $SpO_2$.

These evaluations address the first dimension of RQ1 concerning long-range dependency modeling. The prominent performance gains demonstrate the capacity of LGA's matrix-valued associative memory to capture extended temporal dependencies, supported by the sequence-level normalization that stabilizes optimization over thousands of time steps in full-sequence encoding scenarios.

TABLE IV: Time Series Interpolation and Extrapolation MSE (%, mean$_{\pm \text{std}}$) on PHY and USH Datasets

| Task | Dataset | Metric | Method | | | | | | | | | | $\Delta_{\uparrow}$ |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | mTAND‡ [69] | RKN-$\Delta_t^{\ddagger}$ [61] | GRU-$\Delta_t^{\ddagger}$ [11] | GRU-D‡ [32] | NODE‡ [19] | R-ODE‡ [5] | GRU-ODE-B‡ [68] | CRU‡ [9] | ACSSM‡ [45] | LFormer (ours) | |
| TS-I | PHY | MSE$_{\downarrow}$ | 0.208$_{\pm 0.025}$ | 0.186$_{\pm 0.030}$ | 0.271$_{\pm 0.057}$ | 0.338$_{\pm 0.027}$ | 0.212$_{\pm 0.027}$ | 0.236$_{\pm 0.009}$ | 0.521$_{\pm 0.038}$ | 0.182$_{\pm 0.091}$ | <u>0.116</u>$_{\pm 0.011}$ | **0.040**$_{\pm 0.009}$ | +65.34% |
| | USH | MSE$_{\downarrow}$ | 1.766$_{\pm 0.009}$ | 0.009$_{\pm 0.002}$ | 0.090$_{\pm 0.059}$ | 0.944$_{\pm 0.011}$ | 1.798$_{\pm 0.009}$ | 0.831$_{\pm 0.008}$ | 0.841$_{\pm 0.142}$ | 0.016$_{\pm 0.006}$ | <u>0.006</u>$_{\pm 0.001}$ | **0.001**$_{\pm 0.000}$ | +83.33% |
| TS-E | PHY | MSE$_{\downarrow}$ | **0.340**$_{\pm 0.020}$ | 0.703$_{\pm 0.050}$ | 0.870$_{\pm 0.077}$ | 0.873$_{\pm 0.071}$ | 0.725$_{\pm 0.072}$ | 0.467$_{\pm 0.006}$ | 0.798$_{\pm 0.071}$ | 0.629$_{\pm 0.093}$ | 0.627$_{\pm 0.019}$ | <u>0.372</u>$_{\pm 0.002}$ | -9.41% |
| | USH | MSE$_{\downarrow}$ | 2.360$_{\pm 0.038}$ | 1.491$_{\pm 0.272}$ | 2.081$_{\pm 0.054}$ | 1.718$_{\pm 0.015}$ | 2.034$_{\pm 0.005}$ | 1.955$_{\pm 0.466}$ | 5.437$_{\pm 1.020}$ | 1.273$_{\pm 0.066}$ | <u>0.941</u>$_{\pm 0.014}$ | **0.840**$_{\pm 0.018}$ | +10.73% |

Note: The best results are highlighted in **bold**, and the second best are <u>underlined</u>. $\Delta$ denotes the relative performance gain of LFormer over the best-performing baseline. The arrows $\uparrow$ and $\downarrow$ indicate whether a higher or lower value is better. Results marked with $^{\ddagger}$ are taken from [45].




(a) PTS-C Accuracy on the HA dataset

(b) PTS-R MSE on the PDL dataset

Fig. 4: Comparison of per-point time series classification and regression performance. Baseline results are reported from [45].

*2) Fine-grained Dynamics Tracking (PTS-C and PTS-R) :* Figure 4 reports the point-wise state tracking performance under the non-uniformly sampled benchmarks. LFormer delivers a classification accuracy of 92.9% on the HA dataset (PTS-C) and an MSE of 2.1‰ on the PDL dataset (PTS-R).

On the HA dataset, LFormer yields a modest 1.6% accuracy improvement over the state-of-the-art ACSSM, indicating that both methods achieve comparable performance when categorical decision boundaries dominate over high-fidelity continuous-time integration.

On the PDL dataset, which demands dense tracking of continuous pendulum angles from strongly perturbed image sequences, LFormer achieves a substantial 31.2% relative MSE reduction over ACSSM. Since both architectures employ the identical convolutional embedder, the pronounced performance gap isolates the algorithmic gain from visual feature extraction variations, attributing it to the underlying state transition mechanism. Crucially, our ablation analysis (Table VI) reveals that fixing $\mu = 0.5$ (recovering the standard trapezoidal rule) outperforms the learnable variant. This indicates that under sparse and noisy conditions, unconstrained interpolation can be sensitive to local noise fluctuations. Conversely, the symmetric smoothing prior functions as an inductive low-pass constraint that suppresses noise propagation. Consequently, these gains stem not from any single component, but from the interplay of LGA's matrix memory, cumulative gate stabilization, and task-appropriate smoothing priors.

*3) Trajectory Reconstruction (TS-I and TS-E) :* We evaluate trajectory reconstruction on two scenarios: real-world irregular records (PHY and USH) and controlled synthetic spirals (2DS$_{\alpha}$) under varying signal-to-noise (SNR) ratios.

*a) Real-world Irregular Data:* As reported in Table IV, LFormer achieves substantial gains on TS-I for both PHY and USH, reducing MSE by 65.3% and 83.3% respectively over the strongest baseline (ACSSM). These margins indicate that coupling state transitions with observed time intervals provides an effective continuous-time inductive bias for interpolation. Within this non-causal setting, LGA leverages bidirectional temporal gaps to modulate the associative memory, circumventing the drift and error accumulation that solver-dependent methods can suffer under sparse conditions.

The TS-E results reveal an architectural boundary: LFormer reduces MSE by 10.73% over ACSSM on USH, yet trails mTAND by 9.41% on PHY. This divergence reflects that LGA's recurrent memory excels on the smooth dynamics of USH, whereas mTAND's query-based attention pooling is better suited to the non-stationary signals of PHY.

The ablation results support this: on USH, removing the decoupled multi-head gating or MHLGA incurs the largest performance drop (MSE rises to 0.890 and 0.884; Table VI). This highlights that head-wise specialization across distinct temporal scales is central to capturing the multi-frequency structure of climate variables.

In contrast, on PHY, all ablated variants stay within a narrow MSE range (0.370–0.379), with several configurations matching or slightly surpassing the full model (0.372). This insensitivity indicates that the extrapolation bottleneck of LFormer on PHY stems from a system-level constraint. When handling highly non-stationary signals, extrapolation demands acute sensitivity to localized boundary fluctuations. mTAND directly queries sparse past evidence at arbitrary future timestamps, avoiding the forward drift inherent in the recurrent state-transition of LGA.

TABLE V: Time Series Interpolation and Extrapolation MAE and RMSE (%, mean$_{\pm \text{std}}$) on Three $\texttt{2DS}_\alpha$ Datasets

| Task | $\alpha$ | Metric | Method | | | | | | | $\Delta_\uparrow$ |
|---|---|---|---|---|---|---|---|---|---|---|
| | | | Transformer$^{\S}$ [13] | NODE$^{\S}$ [19] | R-ODE$^{\S}$ [19] | O-ODE$^{\S}$ [5] | NCDE$^{\S}$ [20] | ContiFormer$^{\S}$ [10] | LFormer (ours) | |
| TS-I | 2 | MAE$_\downarrow$ | $2.33_{\pm 0.60}$ | $2.49_{\pm 0.30}$ | $2.88_{\pm 0.54}$ | $4.12_{\pm 1.00}$ | $2.96_{\pm 0.23}$ | $\underline{0.41}_{\pm 0.07}$ | $\mathbf{0.40}_{\pm 0.06}$ | +2.44% |
| | | RMSE$_\downarrow$ | $3.75_{\pm 0.50}$ | $2.10_{\pm 0.21}$ | $3.04_{\pm 0.48}$ | $4.49_{\pm 1.26}$ | $2.92_{\pm 0.32}$ | $\underline{0.45}_{\pm 0.11}$ | $\mathbf{0.43}_{\pm 0.09}$ | +4.44% |
| | 0.2 | MAE$_\downarrow$ | $1.75_{\pm 0.18}$ | $1.58_{\pm 0.12}$ | $3.16_{\pm 0.43}$ | $4.73_{\pm 1.21}$ | $4.58_{\pm 0.93}$ | $\underline{0.42}_{\pm 0.01}$ | $\mathbf{0.33}_{\pm 0.02}$ | +21.43% |
| | | RMSE$_\downarrow$ | $2.30_{\pm 0.51}$ | $1.88_{\pm 0.09}$ | $3.34_{\pm 0.10}$ | $4.82_{\pm 1.97}$ | $4.60_{\pm 1.08}$ | $\underline{0.42}_{\pm 0.04}$ | $\mathbf{0.34}_{\pm 0.02}$ | +19.05% |
| | 0.02 | MAE$_\downarrow$ | $1.42_{\pm 0.14}$ | $1.90_{\pm 0.17}$ | $1.95_{\pm 0.26}$ | $4.25_{\pm 1.48}$ | $5.12_{\pm 0.47}$ | $\underline{0.53}_{\pm 0.07}$ | $\mathbf{0.33}_{\pm 0.09}$ | +37.74% |
| | | RMSE$_\downarrow$ | $1.37_{\pm 0.15}$ | $1.90_{\pm 0.17}$ | $2.09_{\pm 0.22}$ | $4.53_{\pm 1.34}$ | $4.80_{\pm 0.50}$ | $\underline{0.50}_{\pm 0.06}$ | $\mathbf{0.31}_{\pm 0.06}$ | +38.00% |
| TS-E | 2 | MAE$_\downarrow$ | $4.48_{\pm 0.82}$ | $4.84_{\pm 0.43}$ | $4.46_{\pm 0.60}$ | $4.29_{\pm 1.34}$ | $3.32_{\pm 0.85}$ | $\underline{0.69}_{\pm 0.07}$ | $\mathbf{0.53}_{\pm 0.06}$ | +23.19% |
| | | RMSE$_\downarrow$ | $5.36_{\pm 0.97}$ | $6.02_{\pm 0.58}$ | $5.70_{\pm 0.57}$ | $4.68_{\pm 1.70}$ | $3.44_{\pm 1.06}$ | $\underline{0.75}_{\pm 0.07}$ | $\mathbf{0.52}_{\pm 0.06}$ | +30.67% |
| | 0.2 | MAE$_\downarrow$ | $2.79_{\pm 1.11}$ | $3.35_{\pm 0.43}$ | $4.02_{\pm 0.59}$ | $4.72_{\pm 1.96}$ | $3.37_{\pm 0.78}$ | $\underline{0.69}_{\pm 0.09}$ | $\mathbf{0.44}_{\pm 0.04}$ | +36.23% |
| | | RMSE$_\downarrow$ | $2.96_{\pm 1.46}$ | $4.38_{\pm 0.35}$ | $4.78_{\pm 0.15}$ | $5.03_{\pm 1.97}$ | $3.81_{\pm 1.02}$ | $\underline{0.74}_{\pm 0.10}$ | $\mathbf{0.45}_{\pm 0.06}$ | +39.19% |
| | 0.02 | MAE$_\downarrow$ | $1.49_{\pm 0.12}$ | $2.02_{\pm 0.07}$ | $1.52_{\pm 0.09}$ | $2.83_{\pm 0.88}$ | $2.38_{\pm 0.04}$ | $\underline{0.65}_{\pm 0.08}$ | $\mathbf{0.34}_{\pm 0.08}$ | +47.69% |
| | | RMSE$_\downarrow$ | $1.37_{\pm 0.11}$ | $2.07_{\pm 0.02}$ | $1.59_{\pm 0.05}$ | $2.69_{\pm 0.82}$ | $2.24_{\pm 0.03}$ | $\underline{0.64}_{\pm 0.10}$ | $\mathbf{0.32}_{\pm 0.06}$ | +50.00% |

Note: The best results are highlighted in **bold**, and the second best are underlined. $\Delta$ denotes the relative performance gain of LFormer over the best-performing baseline. The arrows $\uparrow$ and $\downarrow$ indicate whether a higher or lower value is better. Results marked with $^{\S}$ are taken from [10] under identical evaluation protocols.

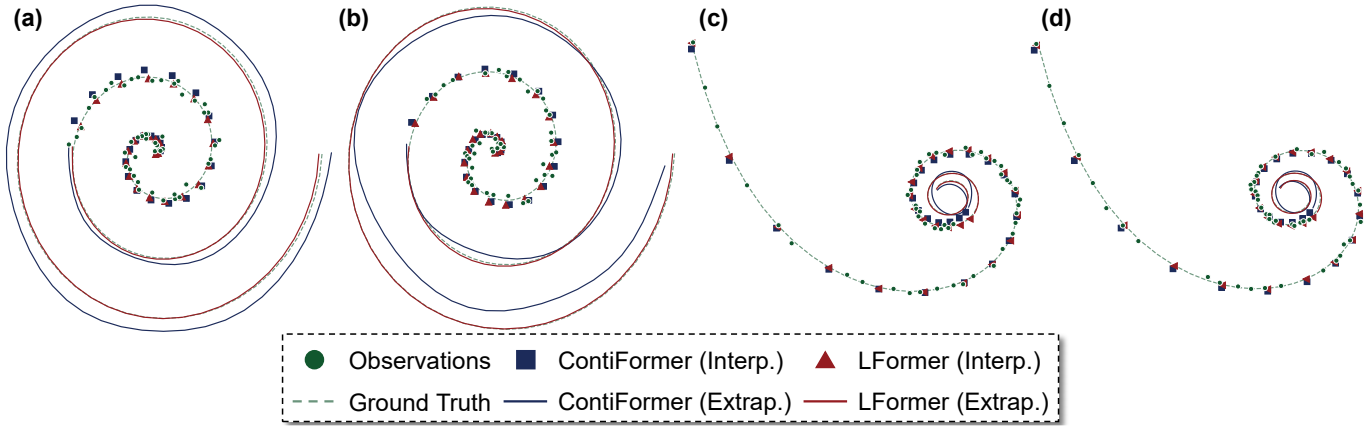



Fig. 5: Qualitative trajectory reconstruction comparison on the $\texttt{2DS}_{\alpha=0.02}$ benchmark. Panels (a)-(b) and panels (c)-(d) show standard Archimedean spirals and nonlinear asymptotic spirals, respectively. The initial temporal half contains 30 randomly sampled irregular observations (green dots) for interpolation, while the terminal half is left unobserved for extrapolation capacity. LFormer (red) closely tracks the continuous ground-truth trajectory (dashed green) during the unobserved extrapolation phase, whereas ContiFormer (blue) exhibits structural divergence.

This trade-off yields a concrete direction for future architectures: integrating LGA with explicit multi-scale timestamp query networks. Such a hybrid approach may effectively reconcile global trajectory consistency (evidenced by TS-E results on USH and real-world TS-I benchmarks) and acute localized temporal awareness in unobserved regimes.

*b) Synthetic Irregular Trajectories:* As detailed in Table V, LFormer achieves the best MAE and RMSE on all three $\texttt{2DS}_\alpha$ variants in both TS-I and TS-E. The margin over ContiFormer, the strongest baseline, widens as $\alpha$ decreases: from marginal 2.44% (TS-I) MAE reductions at $\alpha = 2$ to pronounced 37.7% (TS-I) and 47.7% (TS-E) MAE reductions at $\alpha = 0.02$. Note that both models employ the same continuous time embedding on this benchmark [10]; the widening gap therefore reflects differences in the underlying state transition mechanisms.

On the $\texttt{2DS}_\alpha$ benchmark, $\alpha$ controls the geometric variance of generated trajectories under a constant observational noise. At $\alpha = 2$, the signal dominates the noise, and all models can reliably recover the underlying trajectory. As $\alpha$ decreases to $0.02$, the signal weakens and the trajectories become noise-dominated. Under these low SNR conditions, solver-dependent methods can suffer from high-frequency overshooting when fitting sparse and noisy observations (as shown in Fig. 5).

We attribute this to the bounded liquid gate: as a convex combination within $(0, 1)$, the formulation limits the propagation of high-frequency noise across time steps. Rather than acting as a spectral filter, this formulation introduces an inductive smoothness prior under strong noise, consistent with the low-pass characteristics of exponential moving averages.

### C. Component Analysis

To evaluate the contributions of individual components in LFormer, we conduct an ablation study across all benchmark datasets. Taking the complete LFormer as the baseline (*Full*), variants are organized into three functional groups:

- **Macroscopic Architecture** evaluates structural configurations via three variants
  - *w/o MHLGA* substitutes the MHLGA module with standard multi-head self-attention [13].

TABLE VI: Comprehensive Ablation Study across Real-world and Synthetic Benchmarks (mean$_{\pm \text{std}}$)

| Task | Dataset | Metric | Full | Macroscopic Architecture | | | Liquid Gate Dynamics | | | Stabilization |
|---|---|---|---|---|---|---|---|---|---|---|
| | | | LFormer | w/o MHLGA | w/o SwiGLU | w/o Output Gate | w/o Decoupled $\mu$ | w/o Learnable $\mu$ | w/o Input-aware $\mathbf{g}$ | w/o Normalized $\mathbf{g}$ |
| LTS-R | HR | RMSE$_\downarrow$ | 1.52$_{\pm 0.07}$ | 1.93$_{\pm 0.20}$ | 1.55$_{\pm 0.15}$ | 1.57$_{\pm 0.20}$ | **1.50**$_{\pm 0.11}$ | 1.60$_{\pm 0.17}$ | 1.68$_{\pm 0.10}$ | 1.52$_{\pm 0.06}$ |
| | RR | | **0.97**$_{\pm 0.02}$ | 1.16$_{\pm 0.04}$ | 1.04$_{\pm 0.05}$ | 1.06$_{\pm 0.09}$ | 1.06$_{\pm 0.04}$ | 1.05$_{\pm 0.07}$ | 1.19$_{\pm 0.02}$ | 1.14$_{\pm 0.06}$ |
| | $SpO_2$ | | 0.42$_{\pm 0.02}$ | 0.48$_{\pm 0.02}$ | **0.41**$_{\pm 0.01}$ | 0.42$_{\pm 0.03}$ | 0.44$_{\pm 0.03}$ | 0.42$_{\pm 0.01}$ | 0.48$_{\pm 0.05}$ | 0.45$_{\pm 0.02}$ |
| *Average $\Delta$ vs. Full Model* | | | - | -20.28% | -2.27% | -4.19% | -4.24% | -4.50% | -15.83% | -8.22% |
| LTS-C | EC | Acc.$_\uparrow$ (%) | **45.30**$_{\pm 3.50}$ | 39.70$_{\pm 2.60}$ | 41.80$_{\pm 5.40}$ | 42.00$_{\pm 5.20}$ | 40.30$_{\pm 4.40}$ | 40.30$_{\pm 4.40}$ | 41.00$_{\pm 4.10}$ | 39.70$_{\pm 4.60}$ |
| | EW | | **89.40**$_{\pm 3.20}$ | 83.30$_{\pm 1.80}$ | 87.20$_{\pm 2.20}$ | 86.70$_{\pm 4.10}$ | 88.90$_{\pm 4.60}$ | 88.90$_{\pm 4.60}$ | 88.90$_{\pm 1.80}$ | 88.30$_{\pm 4.40}$ |
| | HB | | 81.60$_{\pm 1.60}$ | 80.30$_{\pm 1.60}$ | 80.00$_{\pm 1.60}$ | 81.00$_{\pm 2.10}$ | 80.30$_{\pm 0.60}$ | 80.30$_{\pm 0.60}$ | 80.30$_{\pm 3.30}$ | **82.30**$_{\pm 2.30}$ |
| | MI | | **65.30**$_{\pm 7.00}$ | 63.50$_{\pm 5.80}$ | 61.40$_{\pm 6.30}$ | 59.60$_{\pm 6.40}$ | 60.40$_{\pm 6.00}$ | 60.40$_{\pm 6.00}$ | 61.80$_{\pm 5.50}$ | 60.00$_{\pm 4.10}$ |
| | SCP1 | | 84.50$_{\pm 1.70}$ | **89.20**$_{\pm 4.00}$ | 84.90$_{\pm 3.70}$ | 85.20$_{\pm 3.50}$ | 84.00$_{\pm 3.80}$ | 84.50$_{\pm 3.70}$ | 86.40$_{\pm 3.50}$ | 86.40$_{\pm 3.50}$ |
| | SCP2 | | 61.80$_{\pm 3.00}$ | 63.20$_{\pm 3.30}$ | 62.80$_{\pm 2.60}$ | **66.00**$_{\pm 2.40}$ | 62.10$_{\pm 2.90}$ | 61.80$_{\pm 3.40}$ | 63.50$_{\pm 3.60}$ | 61.80$_{\pm 3.40}$ |
| *Average $\Delta$ vs. Full Model* | | | - | -2.62% | -2.67% | -2.02% | -3.47% | -3.45% | -2.00% | -3.10% |
| TS-I | PHY | MSE$_\downarrow$ (%) | 0.040$_{\pm 0.015}$ | 0.070$_{\pm 0.028}$ | **0.033**$_{\pm 0.005}$ | 0.042$_{\pm 0.003}$ | 0.044$_{\pm 0.004}$ | 0.047$_{\pm 0.018}$ | 0.047$_{\pm 0.007}$ | 0.132$_{\pm 0.053}$ |
| | USH | | **0.001**$_{\pm 0.000}$ | 0.003$_{\pm 0.002}$ | **0.001**$_{\pm 0.000}$ | **0.001**$_{\pm 0.000}$ | **0.001**$_{\pm 0.000}$ | **0.001**$_{\pm 0.000}$ | **0.001**$_{\pm 0.000}$ | inf$_{\pm \text{nan}}$ |
| | $2DS_{\alpha=2}$ | | 0.43$_{\pm 0.09}$ | 0.48$_{\pm 0.07}$ | 0.50$_{\pm 0.19}$ | 0.45$_{\pm 0.07}$ | **0.42**$_{\pm 0.09}$ | 0.46$_{\pm 0.10}$ | 0.52$_{\pm 0.18}$ | 0.45$_{\pm 0.02}$ |
| | $2DS_{\alpha=0.2}$ | RMSE$_\downarrow$ (%) | **0.34**$_{\pm 0.02}$ | 0.45$_{\pm 0.09}$ | 0.38$_{\pm 0.05}$ | 0.38$_{\pm 0.04}$ | 0.39$_{\pm 0.07}$ | 0.37$_{\pm 0.05}$ | 0.39$_{\pm 0.10}$ | 0.39$_{\pm 0.07}$ |
| | $2DS_{\alpha=0.02}$ | | 0.31$_{\pm 0.06}$ | 0.42$_{\pm 0.04}$ | 0.37$_{\pm 0.08}$ | 0.40$_{\pm 0.06}$ | 0.31$_{\pm 0.04}$ | **0.30**$_{\pm 0.04}$ | 0.32$_{\pm 0.06}$ | 0.34$_{\pm 0.04}$ |
| *Average $\Delta$ vs. Full Model* | | | - | -41.22% | -11.75% | -11.41% | -5.43% | -6.18% | -12.30% | -32.88% |
| TS-E | PHY | MSE$_\downarrow$ (%) | 0.372$_{\pm 0.002}$ | 0.379$_{\pm 0.002}$ | **0.370**$_{\pm 0.003}$ | 0.375$_{\pm 0.005}$ | **0.370**$_{\pm 0.005}$ | 0.374$_{\pm 0.006}$ | 0.373$_{\pm 0.004}$ | **0.370**$_{\pm 0.002}$ |
| | USH | | 0.840$_{\pm 0.018}$ | 0.884$_{\pm 0.007}$ | **0.835**$_{\pm 0.010}$ | 0.836$_{\pm 0.005}$ | 0.890$_{\pm 0.116}$ | 0.837$_{\pm 0.014}$ | 0.842$_{\pm 0.004}$ | inf$_{\pm \text{nan}}$ |
| | $2DS_{\alpha=2}$ | | 0.52$_{\pm 0.06}$ | 0.56$_{\pm 0.08}$ | 0.51$_{\pm 0.08}$ | 0.55$_{\pm 0.10}$ | **0.49**$_{\pm 0.05}$ | 0.52$_{\pm 0.08}$ | 0.57$_{\pm 0.08}$ | 0.51$_{\pm 0.08}$ |
| | $2DS_{\alpha=0.2}$ | RMSE$_\downarrow$ (%) | 0.45$_{\pm 0.06}$ | 0.50$_{\pm 0.02}$ | 0.47$_{\pm 0.04}$ | 0.49$_{\pm 0.09}$ | 0.47$_{\pm 0.04}$ | 0.47$_{\pm 0.07}$ | **0.44**$_{\pm 0.07}$ | 0.46$_{\pm 0.06}$ |
| | $2DS_{\alpha=0.02}$ | | **0.32**$_{\pm 0.06}$ | 0.43$_{\pm 0.01}$ | 0.43$_{\pm 0.10}$ | 0.50$_{\pm 0.21}$ | 0.33$_{\pm 0.05}$ | 0.33$_{\pm 0.04}$ | 0.35$_{\pm 0.06}$ | 0.37$_{\pm 0.05}$ |
| *Average $\Delta$ vs. Full Model* | | | - | -11.64% | -7.14% | -14.65% | -1.33% | -1.68% | -3.38% | -6.44% |
| PTS-C | HA | Acc.$_\uparrow$ (%) | **92.87**$_{\pm 0.16}$ | 89.94$_{\pm 0.76}$ | 90.87$_{\pm 0.49}$ | 91.72$_{\pm 0.35}$ | 91.83$_{\pm 0.17}$ | 91.81$_{\pm 0.17}$ | 91.80$_{\pm 0.38}$ | 20.49$_{\pm 0.00}$ |
| *$\Delta$ vs. Full Model* | | | - | -3.15% | -2.15% | -1.24% | -1.12% | -1.14% | -1.15% | Collapse |
| PTS-R | PDL | MSE$_\downarrow$ (‰) | 2.05$_{\pm 0.25}$ | inf$_{\pm \text{nan}}$ | inf$_{\pm \text{nan}}$ | inf$_{\pm \text{nan}}$ | inf$_{\pm \text{nan}}$ | **1.56**$_{\pm 0.36}$ | 2.85$_{\pm 0.26}$ | inf$_{\pm \text{nan}}$ |
| *$\Delta$ vs. Full Model* | | | - | - | - | - | - | +23.90% | -39.02% | - |

Note: 2DS denotes the Spiral 2D dataset evaluated under different $\alpha$ values. The arrows $\uparrow$ and $\downarrow$ indicate whether a higher or lower value is better. **$\Delta$ vs. Full Model** denotes the average relative percentage difference in performance compared to the full LFormer model. **inf$_{\pm \text{nan}}$** indicates the training process collapsed due to "NaN" or "Inf" values. **Collapse** denotes performance degradation by orders of magnitude.

  - *w/o SwiGLU* replaces the SwiGLU channel mixer with a standard Feed-Forward Network [13].
  - *w/o Output Gate* removes the output gate.
- **Liquid Gate Dynamics** dissect continuous-time transition equations via three configurations:
  - *w/o Decoupled* $\mu$ shares a global $\mu$ across all multi-head subspaces instead of learning head-specific coefficients independently.
  - *w/o Learnable* $\mu$ fixes the learnable interpolation weight as $0.5$, recovering the trapezoidal rule.
  - *w/o Input-aware* $\mathbf{g}$ replaces input-driven gating with a deterministic time-decay.
- **Stabilization** assesses optimization stability via *w/o Normalized* $\mathbf{g}$, which ablates the sequence-level normalization from the cumulative gate computation.

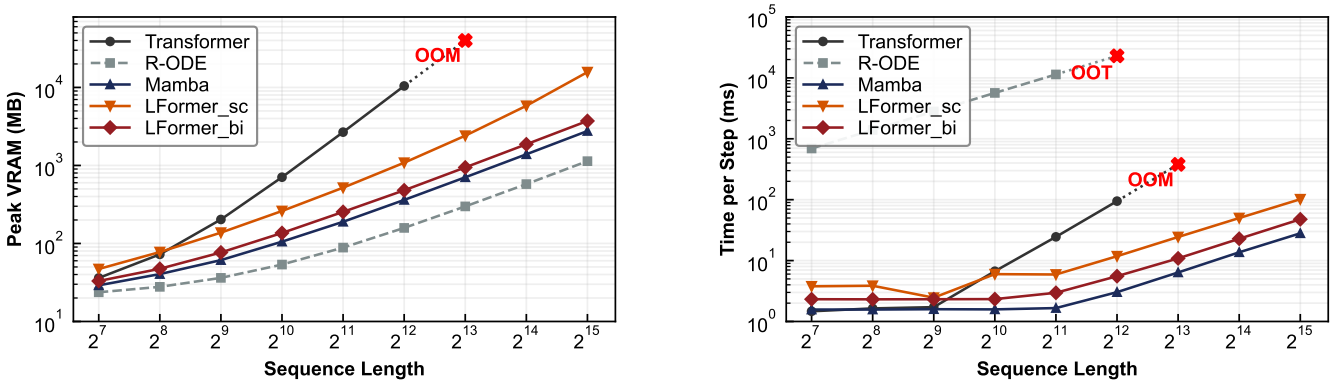
Empirical evaluation metrics for these structural configurations are compiled in Table VI, with specialized architectural analysis detailed below.

*1) Macroscopic Architecture:*

- **MHLGA is critical for regression tasks under irregular sampling.** On classification tasks, *w/o MHLGA* and *w/o SwiGLU* result in comparable, minor degradation (LTS-C: -2.62% vs. -2.67%; PTS-C: -3.15% vs. -2.15%). This suggests coarse-grained categorical decisions are relatively robust to the specific choices of temporal encoders or channel mixers. On regression tasks, this gap widens substantially: *w/o MHLGA* degrades performance far more severely than *w/o SwiGLU* on both LTS-R (-20.28% vs. -2.27%) and TS-I (-41.22% vs. -11.75%) tasks. Furthermore, on the highly sparse PDL dataset, all macroscopic variants completely collapse, underscoring the necessity of the full architectural design.
- **Output gating supports extrapolation and sparse-regime stability.** Removing the output gate causes the most severe drop among macroscopic variants on TS-E (-14.65%) and leads to training collapse on PDL. Conversely, its impact remains moderate on classification and dense regression tasks (LTS-R: -4.19%; LTS-C: -2.02%). This aligns with the output gate's function as a nonlinear filter for retrieved memory. In sparse or extrapolative scenarios where retrieved states are unreliable, explicitly suppressing noisy channels is crucial; under dense sampling, the underlying liquid gating mechanism already provides sufficient implicit filtering.

*2) Liquid Gate Dynamics:*

- **Decoupled $\mu$ enables subspace diversity.** *w/o Decoupled* $\mu$ degrades performance consistently across all tasks and causes training collapse on PTS-R. This highlights the importance of head-specific interpolation for capturing multi-scale temporal dynamics.
- **Learnable $\mu$ balances adaptability and stability.** Fixing $\mu$ (*w/o Learnable* $\mu$) degrades performance across most tasks, yet yields a 23.90% MSE improvement on PTS-R. This suggests that under highly irregular, sparse observations, unconstrained learned interpolation can overfit to local noise, whereas the symmetric trapezoidal prior suppresses error accumulation at the cost of local flexibility.
- **Deterministic temporal decay fails across complex vector fields.** Replacing input-driven modulation with a static time-decay (*w/o Input-aware* $\mathbf{g}$) causes severe




(a) Peak GPU Memory Usage (b) Per-Step Latency

Fig. 6: Empirical complexity scaling. (a) Peak GPU memory usage and (b) training step latency as functions of sequence length. OOM (Out-Of-Memory) indicates that execution exceeded available GPU memory; OOT (Out-Of-Time) indicates that a single step exceeded the measurement time limit.

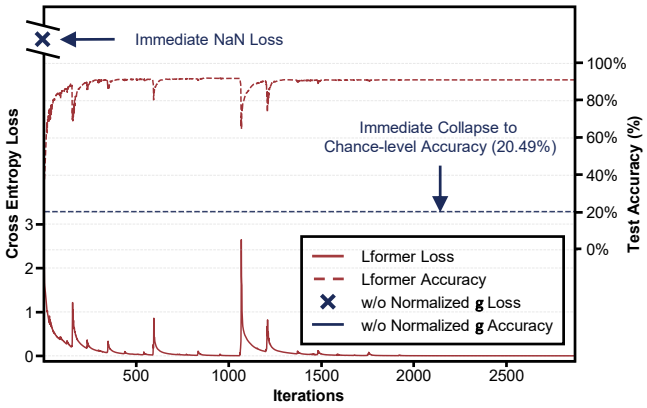



Fig. 7: Optimization dynamics during the stabilization ablation on the PTS-C benchmark. The left and right vertical axes track training loss convergence (solid curves) and out-of-sample test accuracy (dashed curves), respectively.

degradation on regression tasks (LTS-R: -15.83%; TS-I: -12.30%; PTS-R: -39.02%), whereas classification performance is largely unaffected (LTS-C: -2.00%; PTS-C: -1.15%). This indicates that input-aware gating is critical for distinguishing temporal structure from noise when modeling continuous-time dynamics.

*3) Numerical Stabilization:* Removing sequence-level normalization degrades average performance across LTS-R (-8.22%), LTS-C (-3.10%), TS-I (-32.88%), and TS-E (-6.44%), with individual exceptions on `HB` and `PHY` (TS-E) where it proves unnecessary or mildly beneficial. This omission causes training collapse on PTS-C, PTS-R, and on `USH` in both TS-I and TS-E. Figure 7 illustrates the training dynamics on PTS-C: *w/o Normalized* $\mathbf{g}$ suffers immediate gradient explosion and flatlines near the random-guessing level, while LFormer maintains stable optimization and smooth convergence.

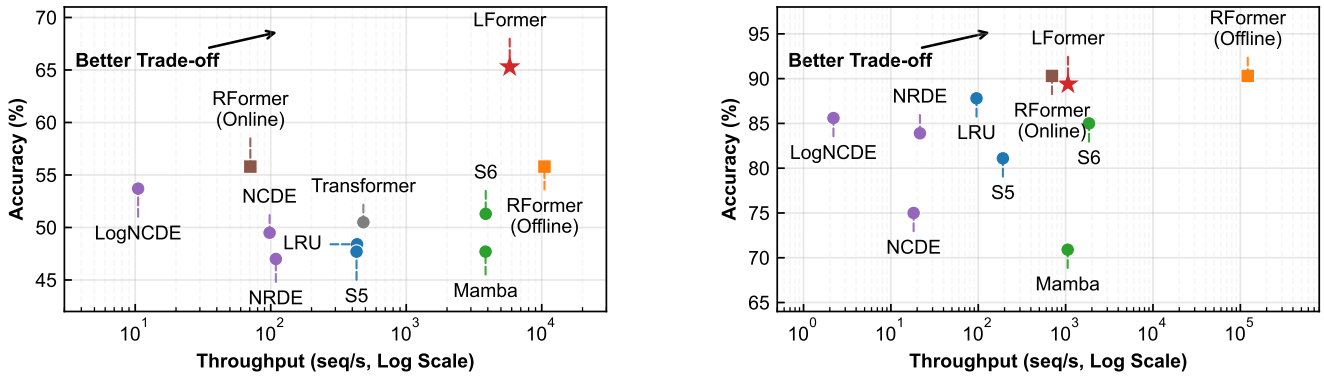
The failure stems from the unconstrained cumulative sum in the liquid gate. Without normalization, the prefix sum over non-negative decay terms grows monotonically with sequence length, and the subsequent exponentiation amplifies this into numerical overflow or vanishing gradients. By bounding the cumulative gate, sequence-level normalization prevents these extremes. This mechanism is therefore essential for stable training on long or irregularly sampled sequences, while on shorter or regularly sampled tasks its role can be partially compensated by other components.

### D. Computational Efficiency (RQ3)

To address RQ3, we evaluate LFormer across three dimensions: (1) empirical complexity scaling with sequence length; (2) hardware utilization and inference throughput; and (3) the accuracy-efficiency Pareto trade-off.

*1) Empirical Complexity Scaling:* To quantify asymptotic complexity, we evaluate two variants of the parallel LGA operator: a causal variant $\text{LFormer}_{\text{sc}}$ that leverages a prefix scan (Eq. (18)), and a non-causal bidirectional variant $\text{LFormer}_{\text{bi}}$ that uses full-sequence matrix multiplication (Eq. (19)). We benchmark both variants against the standard Transformer [13], Mamba [6], and R-ODE [5]. Synthetic trajectories are generated with a fixed batch size of 8 and a feature dimension of 12, while the sequence length scales exponentially from $2^6$ to $2^{15}$ steps. For fair comparison, we align hyperparameters across all models: hidden dimension 64, depth 2, and 4 attention heads for attention-based variants.

Fig. 6a reports the peak VRAM consumption, defined as the maximum GPU memory allocated during a complete training iteration. The Transformer exhibits quadratic memory growth and exceeds GPU capacity at $2^{13}$ steps (OOM). Both $\text{LFormer}_{\text{sc}}$ and $\text{LFormer}_{\text{bi}}$ exhibit linear memory scaling across all tested lengths, where $\text{LFormer}_{\text{sc}}$ incurs higher memory overhead due to intermediate state maintenance during the prefix scan. Both LFormer variants show a linear spatial scaling profile, avoiding the quadratic bottleneck of global attention. Fig. 6b reports the execution latency, measured as the average time over 10 consecutive training forward-backward steps. Due to sequential numerical integration, R-ODE incurs substantial overhead, exceeding $10^4$ ms at sequence length $2^{11}$. We mark this as out-of-time (OOT) and omit longer




(a) Pareto Frontier on the MI Dataset

(b) Pareto Frontier on the EW Dataset

Fig. 8: The efficiency-accuracy Pareto frontier on representative LTS-C datasets. The x-axis shows inference throughput (sequences per second) on a logarithmic scale; the y-axis shows classification accuracy.

TABLE VII: Parameter Count and Inference Throughput

| **Method** | $\lvert\theta\rvert \downarrow$ | $\mathcal{T} \uparrow$ | $\Delta \uparrow$ |
|---|---|---|---|
| R-ODE [5] | **34.56** | 2.46 | 1.0× |
| Transformer [13] | 101.38 | 257.95 | ∼105× |
| Mamba [6] | <u>66.69</u> | **8,974.65** | ∼**3,648**× |
| **LFormer (ours)** | 103.86 | <u>4,652.90</u> | ∼1,891× |

Note: The best results are highlighted in **bold**, and the second best are <u>underlined</u>. The arrows ↑ and ↓ indicate whether a higher or lower value is better. $\lvert\theta\rvert$ denotes the number of trainable parameters (K), $\mathcal{T}$ represents the inference throughput (sequence per second), and $\Delta$ indicates the speedup relative to the R-ODE baseline.

sequences. Both LFormer variants show a linear temporal scaling profile. Despite its native PyTorch implementation, LFormer closely matches Mamba which is accelerated by the hardware-optimized selective scan.

These empirical observations confirm that LFormer exhibits linear scaling complexity across spatial and temporal dimensions under both the non-causal and causal settings.

*2) Throughput and Hardware Efficiency:* We further evaluate the practical hardware efficiency of bidirectional LFormer by measuring parameter count and inference throughput under identical configurations. Synthetic sequences are generated with a batch size of 16, a feature dimension of 12, and a sequence length of $4,096$ steps.

Table VII summarizes the throughput and parameter alignment across the evaluated architectures. LFormer matches the parameter footprint of the standard Transformer while achieving substantially higher throughput, supporting the efficacy of its linear formulation. Although LFormer trails Mamba in absolute inference speed, this gap partly reflects Mamba's use of hardware-optimized parallel scan kernels; LFormer relies on standard PyTorch operators without custom CUDA kernels. Consequently, these statistics highlight the substantial potential of LFormer for future operator-level optimization.

*3) Accuracy vs. Efficiency Trade-off:* To quantify the accuracy-efficiency trade-off, we map empirical Pareto frontiers on the MI and EW datasets. All baselines are instantiated using optimal configurations from their respective literature. For RFormer [25], we evaluate two configurations: an online variant that computes signatures during inference, and an offline variant that uses pre-computed cached signatures.

Fig. 8 plots the classification accuracy against logarithmic inference throughput. An ideal architecture occupies the top-right quadrant, maximizing both accuracy and throughput. The visualization reveals several insights:

- **Solver-dependent Models' Bottlenecks:** NCDE, NRDE, and LogNCDE occupy the lowest efficiency tier. Their reliance on numerical solvers necessitates iterative vector-field evaluations per step, imposing a prohibitive temporal bottleneck that limits scalability.
- **Transformer's Scalability Limits:** The standard Transformer achieves moderate throughput via parallelized attention. However, its quadratic complexity limits scalability over extended sequences, resulting in OOM exceptions on the EW dataset.
- **Linear-Time RNNs' Suboptimality:** While LRU and S5 enhance efficiency via diagonalized recurrent matrices, they deliver lower performance, limiting their position on the accuracy-efficiency frontier.
- **RFormer's Real-time Constraint:** The peak throughput of the offline RFormer variant is enhanced by pre-computed signature transformations, which reduce the effective sequence lengths to 200 (MI) and 10 (EW). This preprocessing step assumes offline access to the full sequence, and the online variant shows lower throughput.
- **LFormer's Pareto Optimality:** LFormer (bidirectional) traces the leading Pareto frontier on both datasets. It achieves the highest accuracy on MI and ranks second on EW dataset, with negligible throughput degradation compared to the fastest models.

These observations position LFormer as an efficient continuous-time framework that combines competitive temporal modeling capacities with high-throughput execution in non-causal full-sequence settings.

## VI. Limitations and Future Work

The current LGA formulation has several limitations. First, the sequence-level normalization bounds the total decay budget for stable training, but it assumes offline full-sequence

access; a prefix-normalized variant would be required for online or causal inference. Second, the learnable endpoint interpolation improves flexibility on most benchmarks, but can overfit under highly sparse and noisy conditions, as evidenced by the PTS-R task where a fixed $\mu = 0.5$ outperforms the learned variant. Third, the constrained extrapolation performance on non-stationary signals reveals the architectural boundary of LGA. These limitations suggest several directions for future work including streaming-compatible normalization, task-adaptive endpoint interpolation and integrating LGA with time-aware query networks.

## VII. Conclusion

This paper presented Liquid Gated Attention (LGA), a solver-free operator that embeds observed temporal intervals into gated linear attention. By unifying a continuous-time gating mechanism derived from liquid time-constant dynamics with fast-weight associative memory and a sequence-level normalization, LGA maintains linear spatio-temporal complexity and empirically stable training over extended horizons. Built upon LGA, LFormer provides a modular backbone for continuous-time representation learning. Across six tasks and sixteen benchmarks, LFormer delivers strong results across three core temporal modeling capabilities: long-range dependency modeling, fine-grained state tracking, and trajectory reconstruction from sparse and noisy observations.

Ultimately, LGA demonstrates how continuous-time dynamical principles can be embedded into parallel sequence architectures, positioning LFormer as an efficient backbone for time series representation learning.

## Appendix A
## Derivation of the Closed-form Integral Solution for 1D LTC

We solve the linear non-autonomous ODE that governs a decoupled scalar LTC state $s(t) \in \mathbb{R}$ under a scalar exogenous input $x(t) \in \mathbb{R}$:

$$\frac{\mathrm{d}s(t)}{\mathrm{d}t} = -\big[w_\tau + f\big(x(t)\big)\big]s(t) + b_\tau f\big(x(t)\big), \tag{39}$$

where $w_\tau, b_\tau \in \mathbb{R}$ represent learnable scalar parameters.

Following the structural symmetry assumption established in [22], we introduce a balancing inverse time-constant term $w_\tau$ into the bias compartment to symmetrize the algebraic structure:

$$\frac{\mathrm{d}s(t)}{\mathrm{d}t} = -\big[w_\tau + f(x(t))\big]s(t) + \big[w_\tau + f(x(t))\big]b_\tau. \tag{40}$$

The effect of this structural balancing term on the solution has been analyzed in [22], which demonstrated its negligible impact on the approximation fidelity.

By defining the time-variant system rate coefficient as $p(t) \triangleq w_\tau + f\big(x(t)\big)$, we map Eq. (40) onto the canonical form of a first-order linear non-homogeneous ODE:

$$\frac{\mathrm{d}s(t)}{\mathrm{d}t} + p(t)s(t) = p(t)b_\tau. \tag{41}$$

To derive the analytical trajectory of Eq. (41), we define the standard non-autonomous integrating factor $\mu(t)$ as:

$$\mu(t) \triangleq \exp\left[\int_0^t p(u)\,\mathrm{d}u\right]. \tag{42}$$

From the fundamental theorem of calculus, the derivative of the integrating factor is:

$$\frac{\mathrm{d}\mu(t)}{\mathrm{d}t} = \mu(t)\frac{\mathrm{d}}{\mathrm{d}t}\left[\int_0^t p(u)\,\mathrm{d}u\right] = \mu(t)p(t). \tag{43}$$

Multiplying both sides of the canonical ODE in Eq. (41) by $\mu(t)$ yields:

$$\mu(t)\frac{\mathrm{d}s(t)}{\mathrm{d}t} + \underbrace{\mu(t)p(t)s(t)}_{s(t)\frac{\mathrm{d}\mu(t)}{\mathrm{d}t}} = \mu(t)p(t)b_\tau. \tag{44}$$

Recognizing the left-hand side as the exact derivative of the product $\mu(t)s(t)$, we condense the equation to:

$$\frac{\mathrm{d}}{\mathrm{d}t}\big[\mu(t)s(t)\big] = \mu(t)p(t)b_\tau. \tag{45}$$

Performing definite integration on Eq. (45) from the initial boundary 0 to the target timestamp $t$, and noting that $\mu(0) = \exp(0) = 1$, we obtain:

$$\mu(t)s(t) - s(0) = b_\tau \int_0^t \mu(v)p(v)\,\mathrm{d}v, \tag{46}$$

where $s(0) \in \mathbb{R}$ denotes the initial state condition.

Given that the exponential integrating factor $\mu(t)$ is strictly positive, multiplying both sides by $\mu^{-1}(t)$ isolates the continuous state trajectory, revealing a nested integral expression:

$$\begin{aligned} s(t) &= s(0)\mu^{-1}(t) \\ &\quad + b_\tau \int_0^t \mu^{-1}(t)\mu(v)p(v)\,\mathrm{d}v \\ &= s(0)\exp\left[-\int_0^t p(u)\,\mathrm{d}u\right] \\ &\quad + b_\tau \underbrace{\int_0^t \exp\left[-\int_v^t p(u)\,\mathrm{d}u\right]p(v)\,\mathrm{d}v}_{\text{Nested Integral Term } n(t)}. \end{aligned} \tag{47}$$

To simplify the nested integral $n(t)$, we apply integration by substitution. Define the auxiliary function $a(v)$ as:

$$a(v) \triangleq \int_v^t p(u)\,\mathrm{d}u. \tag{48}$$

Differentiating with respect to $v$ gives $\mathrm{d}a(v) = -p(v)\,\mathrm{d}v$. The integration limits transform as:

$$a(v) = \begin{cases} 0, & \text{when } v = t, \\ \int_0^t p(u)\,\mathrm{d}u, & \text{when } v = 0. \end{cases} \tag{49}$$

Substituting these relations into $n(t)$ yields:

$$\begin{aligned} n(t) &= \int_{a(0)}^{0} \exp[-a]\big(-\mathrm{d}a\big) \\ &= \int_0^{a(0)} \exp[-a]\,\mathrm{d}a \\ &= \Big[-\exp[-a]\Big]_0^{a(0)} \\ &= 1 - \exp\big[-a(0)\big]. \end{aligned} \tag{50}$$

Substituting the result of Eq. (50) into Eq. (47) reduces the state trajectory to:

$$s(t) = \big[s(0) - b_\tau\big] \exp\big[-a(0)\big] + b_\tau. \tag{51}$$

Recalling that $a(0) = \int_0^t p(u)\,\mathrm{d}u$ and $p(t) = w_\tau + f\big(x(t)\big)$, we obtain the closed-form integral solution:

$$s(t) = \big(s(0) - b_\tau\big) \exp\left[-w_\tau t - \int_0^t f\big(x(u)\big)\,\mathrm{d}u\right] + b_\tau. \tag{52}$$

## APPENDIX B
## ERROR ANALYSIS OF THE LEARNABLE ENDPOINT INTERPOLATION

To evaluate the local truncation error of the proposed interpolation, we first establish the requisite mathematical smoothness. Assuming the underlying trajectory $x(u)$ is smooth, the integrand $\tilde{x}(u) = \sigma(\mathbf{x}(u)\mathbf{w}_f + b_f) \in (0,1)$ is twice continuously differentiable, $\tilde{x} \in C^2[t_{n-1}, t_n]$, over the localized time segment with step size $\delta_n = t_n - t_{n-1}$.

A Taylor expansion of the endpoint value $\tilde{x}(t_n)$ centered at $t_{n-1}$ yields:

$$\tilde{x}(t_n) = \tilde{x}(t_{n-1}) + \delta_n \tilde{x}'\left(t_{n-1}\right) + \frac{\delta_n^2}{2}\tilde{x}''\left(t_{n-1}\right) + \mathcal{O}\left(\delta_n^3\right). \tag{53}$$

The exact definite integral can be expressed via Taylor expansion as:

$$\begin{aligned}\int_{t_{n-1}}^{t_n} \tilde{x}(u)\,\mathrm{d}u = \delta_n \tilde{x}\left(t_{n-1}\right) &+ \frac{\delta_n^2}{2}\tilde{x}'\left(t_{n-1}\right) \\ &+ \frac{\delta_n^3}{6}\tilde{x}''\left(t_{n-1}\right) + \mathcal{O}\left(\delta_n^4\right).\end{aligned} \tag{54}$$

To bypass sequential numerical integration, LGA approximates this integral via a learnable endpoint interpolation, parameterized by a bounded coefficient $\mu = \sigma(\theta) \in (0,1)$:

$$I_{\text{approx}} \triangleq \delta_n \left[\mu \tilde{x}\left(t_{n-1}\right) + (1-\mu)\,\tilde{x}\left(t_n\right)\right]. \tag{55}$$

Substituting the expansion of $\tilde{x}(t_n)$ from Eq. (53) into the numerical surrogate in Eq. (55) yields:

$$\begin{aligned} I_{\text{approx}} &= \delta_n \Big\{ \mu \tilde{x}\left(t_{n-1}\right) \\ &\quad + (1-\mu)\Big[\tilde{x}\left(t_{n-1}\right) + \delta_n \tilde{x}'\left(t_{n-1}\right) \\ &\quad + \frac{\delta_n^2}{2}\tilde{x}''\left(t_{n-1}\right) + \mathcal{O}\left(\delta_n^3\right)\Big]\Big\} \\ &= \delta_n \tilde{x}\left(t_{n-1}\right) + \delta_n^2(1-\mu)\tilde{x}'\left(t_{n-1}\right) \\ &\quad + \frac{\delta_n^3}{2}(1-\mu)\tilde{x}''\left(t_{n-1}\right) + \mathcal{O}\left(\delta_n^4\right). \end{aligned} \tag{56}$$

The local truncation error $E$ is defined as the algebraic difference between the exact integral (Eq. (54)) and the numerical surrogate (Eq. (56)):

$$\begin{aligned} E &= \int_{t_{n-1}}^{t_n} \tilde{x}(u)\,\mathrm{d}u - I_{\text{approx}} \\ &= \left[\frac{1}{2} - (1-\mu)\right]\delta_n^2 \tilde{x}'\left(t_{n-1}\right) \\ &\quad + \left[\frac{1}{6} - \frac{1-\mu}{2}\right]\delta_n^3 \tilde{x}''\left(t_{n-1}\right) + \mathcal{O}\left(\delta_n^4\right) \\ &= \left(\mu - \frac{1}{2}\right)\delta_n^2 \tilde{x}'\left(t_{n-1}\right) \\ &\quad + \left(\frac{\mu}{2} - \frac{1}{3}\right)\delta_n^3 \tilde{x}''\left(t_{n-1}\right) + \mathcal{O}\left(\delta_n^4\right). \end{aligned} \tag{57}$$

Equation (57) characterizes the error under two distinct parameterization regimes:

- **General case ($\mu \neq 0.5$):** The error is dominated by the $\delta_n^2$ term, giving a local truncation error of $\mathcal{O}(\delta_n^2)$. The magnitude of the leading error is proportional to $|\mu - 0.5|$, so that moderate deviations from the midpoint do not induce unbounded error amplification. This property allows the model to adapt the interpolation to data-dependent temporal dynamics while preserving numerical stability.
- **Symmetric case ($\mu = 0.5$):** The coefficient of the leading $\delta_n^2$ term vanishes, and the error algebraically reduces to:

$$E = -\frac{1}{12}\delta_n^3 \tilde{x}''\left(t_{n-1}\right) + \mathcal{O}\left(\delta_n^4\right) = \mathcal{O}\left(\delta_n^3\right). \tag{58}$$

  Under this condition, the interpolation recovers the classical trapezoidal rule with second-order local accuracy. For well-behaved integrands, accumulating this local error over a finite interval yields a global error of $\mathcal{O}(\delta^2)$.

## APPENDIX C
## NUMERICAL BOUNDS AND GRADIENT STABILITY OF SEQUENCE-LEVEL NORMALIZATION

Let $\mathbf{u} = \boldsymbol{\Delta} \odot \overline{\mathbf{x}}$ denote the unnormalized decay vector, where $u_i = \delta_i \overline{x}_i$ for $i \in \{1, \ldots, n\}$. In practice, the temporal intervals are non-negative and the interpolated trajectory satisfies $\overline{x}_i \in (0,1)$. Consequently, the elements are non-negative: $u_i \geq 0$ for all $i$. To prevent unbounded accumulation in the cumulative gate, we apply the sequence-level normalization:

$$\hat{u}_i = \frac{u_i}{\sum_{k=1}^{n} u_k + \epsilon}, \tag{59}$$

where $\epsilon > 0$ is a small stability constant that prevents division by zero. Since $u_i \geq 0$ and $\epsilon > 0$, we have $\hat{u}_i \geq 0$ for all $i$.

Let $s_j \triangleq \sum_{i=1}^{j} \hat{u}_i$ denote the $j$-th element of the cumulative sum vector $\mathbf{s} = \mathrm{CumSum}(\hat{\mathbf{u}})$. Since $\hat{u}_i \geq 0$, the sequence $\{s_j\}_{j=1}^{n}$ is monotonically non-decreasing. The terminal value $s_n$ is:

$$s_n = \sum_{i=1}^{n} \hat{u}_i = \frac{\sum_{i=1}^{n} u_i}{\sum_{k=1}^{n} u_k + \epsilon}. \tag{60}$$

where the denominator strictly exceeds the numerator, yielding $s_n < 1$. Combined with the lower bound $s_j \geq 0$, we obtain $s_j \in [0,1)$ for all $j \in \{1, \ldots, n\}$. Thus, the stabilized

cumulative gate $\dot{g}_{\text{stable},j} = \exp(-s_j) \in (e^{-1}, 1]$ for all $j \in \{1, \ldots, n\}$.

**Remark on temporal semantics.** The normalized gate is not algebraically equivalent to the LTC continuous-time transition. Instead, it functions as a regularized surrogate that bounds the total decay budget while preserving the relative temporal allocation across the sequence. This design is suitable for offline full-sequence representation learning; for online streaming or causal inference, a prefix-normalized variant is required to avoid dependence on future observations.

**Remark on gradient stability.** Beyond bounding the forward-pass values, the sequence-level normalization alters the asymptotic behavior of the gradients during back propagation through time (BPTT). Consider the gradient of the cumulative gate $\dot{g}_{\text{stable},k} = \exp(-s_k)$ with respect to the unnormalized decay term $u_i$ for $i \leq k$. By the chain rule:

$$\frac{\partial \dot{g}_{\text{stable},k}}{\partial u_i} = -\dot{g}_{\text{stable},k} \cdot \frac{\partial s_k}{\partial u_i} = -\dot{g}_{\text{stable},k} \cdot \sum_{j=1}^{k} \frac{\partial \hat{u}_j}{\partial u_i}. \tag{61}$$

The partial derivative of the normalized element $\hat{u}_j$ with respect to $u_i$ is evaluated as:

$$\frac{\partial \hat{u}_j}{\partial u_i} = \begin{cases} \dfrac{\sum_{\ell \neq i} u_\ell + \epsilon}{(\sum_{\ell=1}^{n} u_\ell + \epsilon)^2}, & j = i, \\ -\dfrac{u_j}{(\sum_{\ell=1}^{n} u_\ell + \epsilon)^2}, & j \neq i. \end{cases} \tag{62}$$

Summing these derivatives over $j = 1, \ldots, k$ simplifies to:

$$\sum_{j=1}^{k} \frac{\partial \hat{u}_j}{\partial u_i} = \frac{\sum_{\ell=1}^{n} u_\ell + \epsilon - \sum_{j=1}^{k} u_j}{(\sum_{\ell=1}^{n} u_\ell + \epsilon)^2} \geq \frac{\epsilon}{(\sum_{\ell=1}^{n} u_\ell + \epsilon)^2} > 0, \tag{63}$$

where the inequality holds because $\sum_{j=1}^{k} u_j \leq \sum_{\ell=1}^{n} u_\ell$. Substituting this bound back into the chain rule, we obtain the absolute gradient magnitude:

$$\left| \frac{\partial \dot{g}_{\text{stable},k}}{\partial u_i} \right| = \dot{g}_{\text{stable},k} \cdot \sum_{j=1}^{k} \frac{\partial \hat{u}_j}{\partial u_i} \geq e^{-1} \cdot \frac{\epsilon}{(\sum_{\ell=1}^{n} u_\ell + \epsilon)^2}. \tag{64}$$

This quantitative result reveals a crucial property. Let $S \triangleq \sum_{\ell=1}^{n} u_\ell$ denote the total sum of unnormalized decay terms along the sequence. While the gradient signal does inversely scale with the total sum of the sequence, the attenuation is strictly **polynomial** $\mathcal{O}(S^{-2})$. Such polynomial scaling is manageable and easily mitigated by modern adaptive optimizers. In contrast, for the unnormalized variant, the gradient evaluates to $\frac{\partial \dot{g}_k}{\partial u_i} = -\dot{g}_k$, which decays **exponentially** $\mathcal{O}(e^{-S})$ as the cumulative sum grows. Thus, the normalization mathematically mitigates the exponential vanishing gradient problem, enabling robust optimization across extremely long temporal horizons.

## APPENDIX D
## DATASET DETAILS AND PREPROCESSING PROTOCOLS

### A. UEA Multivariate Time Series Archive

We select six benchmarks from the UEA multivariate time series archive [59]: EigenWorms (EW), EthanolConcentration (EC), Heartbeat (HB), MotorImagery (MI), SelfRegulationSCP1 (SCP1), and SelfRegulationSCP2 (SCP2). Following the selection criteria in [67], we adopt benchmarks with more than 200 instances to ensure reliable evaluation. The selected benchmarks span a broad range of sequence lengths (405 to 17,984 steps), enabling assessment of long-range modeling capacity. Duplicate trajectories are removed following the preprocessing protocols in [21], [67]. The remaining sequences are randomly partitioned into training (70%), validation (15%), and testing (15%). Observation timestamps are normalized to $[0, 1]$.

### B. BIDMC Physiological Database

We use three subsets from the BIDMC database [60]: heart rate (HR), respiratory rate (RR), and blood oxygen saturation ($\text{SpO}_2$). The task predicts a single continuous scalar (the vital sign value) from the full trajectory. The input consists of dual-channel physiological waveforms sampled at 125 Hz. A 32-second sliding window produces dense sequences spanning 4,000 time steps. Each sequence is labeled with the corresponding heart rate, respiratory rate, or blood oxygen saturation level. The remaining sequences are randomly partitioned into training (70%), validation (15%), and testing (15%). Observation timestamps are normalized to $[0, 1]$.

### C. Human Activity Benchmark

The Human Activity (HA) benchmark [5] comprises 12-dimensional time-series recordings acquired from four wearable sensors across five subjects. The objective centers on dense, point-wise classification, mapping each temporal observation to one of seven distinct physical activity categories. Adhering to the protocol in [5], the raw trajectories are processed into 6,554 distinct sequence instances. While each canonical sequence contains 211 steps, a subset of 50 temporal observations is randomly sampled per instance to create irregularly sampled sequences. Timestamps are normalized by a scaling factor of $1/211$, and the dataset is partitioned into 4,194 training, 1,049 validation, and 1,311 testing sequences.

### D. Pendulum Tracking Benchmark

The Pendulum (PDL) dataset [61] consists of sequences containing $24 \times 24$ pixel images, where the dense target at each temporal index corresponds to the angular coordinate $(\sin\theta, \cos\theta)$. Consistent with [9], 4,000 sequences are synthesized. Each baseline trajectory spans 100 uniformly spaced steps, from which 50 frames are randomly sub-sampled to construct an irregular temporal grid. Spatially and temporally correlated noise is introduced to the pixel grids following [61]. Observation timestamps are scaled by a factor of 0.1 following [45]. The data configuration comprises 2,000 training, 1,000 validation, and 1,000 testing instances.

### E. Synthetic 2D Spiral Trajectories

Synthetic 2D spiral trajectories ($\text{2DS}_\alpha$) [10] are generated to evaluate trajectory interpolation (TS-I) and extrapolation (TS-E). Each individual trajectory is stochastically assigned

a clockwise or counter-clockwise orientation, governed by Archimedean parametric equations. The first geometric variation is formulated as:

$$\begin{aligned} x(t) &= (a + bt)\cos(t), \\ y(t) &= (a + bt)\sin(t), \end{aligned} \tag{65}$$

and the alternative formulation is defined via:

$$\begin{aligned} x(t) &= \left(a + \frac{50b}{e - t}\right)\cos(e - t), \\ y(t) &= \left(a + \frac{50b}{e - t}\right)\sin(e - t), \end{aligned} \tag{66}$$

where $a$ and $b$ represent latent structural parameters drawn from normal distributions: $a \sim \mathcal{N}(0, \alpha)$ and $b \sim \mathcal{N}(0.3, \alpha)$. Three distinct benchmarks are established by varying the variance scale parameter $\alpha \in \{2, 0.2, 0.02\}$. Additive Gaussian noise $\mathcal{N}(0, 0.1)$ is added to the training trajectories. Each complete trajectory contains 150 uniformly spaced time steps. To simulate partial observations with irregular sampling, the model receives 30 points randomly drawn from the first half of the trajectory ($t \in [1, 75]$). The task requires interpolating the unobserved positions within this interval and extrapolating the full trajectory over the entirely unobserved second half ($t \in [76, 150]$). For each value of $\alpha$, we generate 300 trajectories, partitioned into 200 training and 100 testing instances.

### F. USHCN Benchmark

The United States Historical Climatology Network (USH) dataset [62] comprises daily measurements from 1,218 weather stations across the United States, covering five variables: precipitation, snowfall, snow depth, and daily minimum and maximum temperature. Following the preprocessing pipeline of [45], we retain a subset of stations with continuous records spanning a four-year period starting from 1990. Stations whose observations end before 1994 or begin after 1990 are excluded, along with measurements flagged with unreliable quality markers. Each variable is standardized independently to zero mean and unit variance, and extreme values exceeding four standard deviations from the mean are treated as missing. To introduce irregular sampling, we randomly subsample 50% of the time points and independently drop 20% of the remaining observations, following the protocol of [45]. Only stations retaining at least 730 observations after subsampling are kept. The processed dataset contains 1,192 stations, which are randomly partitioned into training (60%), validation (20%), and testing (20%) sets at the station level. Observation timestamps are encoded as integer days elapsed since January 1, 1950, and normalized to $[0, 1]$.

### G. PhysioNet Benchmark

The PhysioNet Challenge 2012 dataset (PHY) [63] contains 8,000 multivariate clinical time series collected from intensive care unit (ICU) patients during the first 48 hours after admission. Each record comprises 41 measurements, of which we discard 4 static features (age, gender, height, and ICU type) and retain the remaining 37 time-varying clinical variables, following the preprocessing protocol of [5]. Each variable is independently normalized to $[0, 1]$ via min-max scaling, with statistics computed separately for each data split. The processed records are randomly partitioned into training (60%), validation (20%), and testing (20%) sets at the patient level.

## APPENDIX E
## DETAILED BASELINE SPECIFICATIONS

This section provides the detailed description for all baseline architectures used in the experimental evaluation.

### A. Discrete-Time Baselines

- GRU [11] models sequential dependencies through discrete gated recurrent units over uniform step grids. It compresses past information via interleaved reset and update gates, serving as a baseline for discrete recurrence. GRU-$\Delta_t$ is a variant of GRU which takes additional time intervals as input.
- Transformer [13] employs global dot-product self-attention mechanisms over static discrete tokens. It eliminates sequential dependencies via fully parallel matrix operations, evaluating standard discrete relational modeling without explicit temporal distance metrics.
- LRU [64] linearizes and diagonalizes recurrent layers into decoupled complex-valued hidden states to mitigate optimization bottlenecks over long sequences.
- Mamba (S6) [6] introduces input-dependent, selective linear state-space layers parallelized via a hardware-aware associative scan algorithm. In this setup, Mamba operates over uniform discrete steps, with selection step sizes driven by local token features rather than physical temporal distances.
- GRU-D [32] incorporates the missing mask and time intervals into the GRU architecture, enabling it to capture long-term temporal dependencies while leveraging informative missingness for improved prediction.
- RKN-$\Delta_t$ [61] factorizes the latent state representation to scalar Kalman updates. It employs local linear dynamics for state propagation and provides explicit uncertainty estimates alongside predictions.

### B. Solver-Dependent Continuous-Time Baselines

- NODE [19] parameterizes the latent state vector-field derivative via a neural network, mapping state trajectories through continuous integration solved by adaptive-step numerical ODE solvers. It utilizes the adjoint sensitivity method to achieve constant-memory backpropagation.
- The Latent ODE framework (comprising R-ODE [5] and O-ODE [5]) proposes a continuous-time generative framework for irregularly sampled time series. It utilizes neural ODE variants driven by either a recurrent recognition network (R-ODE) or an observation-aware continuous encoder (O-ODE) to infer latent state trajectories.
- LSDE [66] formulates variational Bayesian inference over continuous latent trajectories governed by stochastic differential equations. It performs path tracking over

homogeneous vector fields through a simplified Kullback-Leibler divergence and parameterized continuous diffusion gradients.
- NCDE [20] extends the neural differential paradigm to controlled differential equations, addressing limitations related to initial conditions. It constructs continuous-time paths from irregularly sampled inputs via cubic spline interpolation to drive the hidden state evolution.
- NRDE [21] combines neural controlled dynamics with rough path theory to process high-frequency, irregularly sampled signals. It maps localized signal intervals into bounded algebraic log-signatures, compressing high-frequency variations into summary statistics to accelerate continuous-time scaling.
- LogNCDE [67] integrates the Log-ODE numerical approximation scheme into the neural rough path paradigm for stiff controlled differential equations, targeting long-range dependencies over extended integration horizons.
- ContiFormer [10] combines the continuous-time representation of neural ODEs with self-attention. It extends the attention formulation into the continuous-time domain to parameterize input-dependent vector fields.
- CRU [9] evolves hidden state according to a linear stochastic differential equation, yielding temporal continuity between hidden states and a gating mechanism that integrates noisy irregular observations.
- GRU-ODE-B [68] combines NODE with a Bayesian update network to handle observations that are irregular in both time and feature dimensions, encoding a continuity prior for the latent process.

### C. Solver-Free Continuous-Time Baselines

- CfC [22] derives a bounded, analytic closed-form integral solution to approximate the dynamics of liquid time-constant networks. It bypasses the numerical integration overhead, enabling efficient continuous-time modeling.
- RFormer [25] enhances self-attention with multi-view algebraic path signatures. It constructs non-sequential continuous representations designed to bypass sequential integration overhead across long temporal trajectories.
- mTAND [69] leverages a continuous multi-time attention embedding kernel to map irregular, variable-length observation coordinates into fixed-dimensional continuous-time representations, preserving non-uniform interval metrics without tracking hidden derivatives.
- S5 [70] formulates structured linear state-space layers over non-uniform grids, driving state evolution analytically via bilinear (trapezoidal) transformations. It leverages a time-varying parallel associative scan to directly process physical time intervals under full parallelization.
- ACSSM [45] leverages multi-marginal Doob's $h$-transform mechanics and stochastic optimal control to construct a simulation-free latent dynamics framework. It integrates parallelized variational inference into a linear architecture, enabling parallel optimization over continuous trajectories.

## APPENDIX F
## DETAILED HYPERPARAMETER AND IMPLEMENTATION CONFIGURATIONS

This section provides the complete architectural specifications and hyperparameter settings for LFormer.

Before detailing the network architectures, we adopt the following notational conventions. `Linear(`$d_{\text{in}}$`,` $d_{\text{out}}$`)` denotes a fully-connected layer with $d_{\text{in}}$ input and $d_{\text{out}}$ output dimensions. `Conv(`$c_{\text{in}}$`,` $c_{\text{out}}$`,` $k$`,` $s$`,` $p$`)` denotes a 2D convolutional layer with $c_{\text{in}}$ input channels, $c_{\text{out}}$ output channels, kernel size $k$, stride $s$, and padding $p$. `MaxPool(`$k$`,` $s$`)` denotes a 2D max-pooling operation with kernel size $k$ and stride $s$.

Across all benchmarks, each Liquid Mixer layer consists of a multi-head liquid gated attention (MHLGA) module followed by a SwiGLU channel mixer, organized in a pre-normalization residual structure. LFormer stacks $L$ such layers, with hidden dimension $d$, $H$ attention heads, and SwiGLU expansion dimension $d_{ff}$, denoted as $L \times$ `[MHLGA(`$d$`,` $H$`)` $\rightarrow$ `SwiGLU(`$d_{ff}$`)]`.

All tasks are optimized using Adam with $\beta_1 = 0.9$, $\beta_2 = 0.999$, no weight decay, and a batch size chosen to maximize GPU memory utilization. Early stopping is applied when the validation metric does not improve for 20 consecutive epochs. The learning rate for each benchmark is listed alongside its architecture below.

- **LTS-R Benchmark:**
  - Embedder: `Linear(2, 64)` $\rightarrow$ `ReLU()` $\rightarrow$ `LayerNorm()`
  - Liquid Mixer: $2 \times$ `[MHLGA(64, 4)` $\rightarrow$ `SwiGLU(176)]`
  - Predictor: `Linear(64, 1)`
  - Learning Rate: `0.001`
- **LTS-C Benchmark:** For `EC`, `EW`, `HB`, and `MI`, we adopt the standard configuration ($d = 64$, $H = 4$, $L = 2$). For `SCP1` and `SCP2`, we use a larger hidden dimension ($d = 128$, $d_{ff} = 352$) with a single attention head ($H = 1$) and a single Liquid Mixer layer ($L = 1$), as this configuration proved more effective for these two datasets.
  - Embedder
    - `Linear(`{`EC: 3, EW: 6, HB: 61, MI: 64`}`, 64)` $\rightarrow$ `ReLU()`
    - `Linear(`{`SCP1: 6, SCP2: 7`}`, 128)` $\rightarrow$ `ReLU()`
  - Liquid Mixer
    - {`EC, EW, HB, MI`}: $2 \times$ `[MHLGA(64, 4)` $\rightarrow$ `SwiGLU(176)]`
    - {`SCP1, SCP2`}: $1 \times$ `[MHLGA(128, 1)` $\rightarrow$ `SwiGLU(352)]`
  - Predictor
    - {`EC, EW, HB, MI`}: `Linear(64,` {`EC: 4, EW: 5, HB: 2, MI: 2`}`)`
    - {`SCP1, SCP2`}: `Linear(128,` {`SCP1: 2, SCP2: 2`}`)`
  - Learning Rate: `0.001`
- **PTS-R Benchmark:** Due to the image-based input ($24 \times 24$ pixels), the Pendulum benchmark employs a

convolutional embedder and a larger hidden dimension ($d = 128$, $d_{ff} = 352$).

- Embedder: `Conv(24, 12, 5, 1, 2)` → `ReLU()` → `MaxPool(2, 2)` → `Conv(12, 12, 3, 2, 1)` → `ReLU()` → `MaxPool(2, 2)` → `Flatten()` → `Linear(108, 128)` → `ReLU()`
- Liquid Mixer: $2 \times$ `[MHLGA(128, 4)` → `SwiGLU(352)]`
- Predictor: `Linear(128, 2)`
- Learning Rate: `0.01`

- **PTS-C Benchmark:** For the dense per-point classification objective on Human Activity, we increase the number of Liquid Mixer layers to $L = 4$ to capture fine-grained state transitions.
  - Embedder: `Linear(12, 64)`→`ReLU()`→`LayerNorm()`→`Linear(64, 64)`
  - Liquid Mixer: $4 \times$ `[MHLGA(64, 4)` → `SwiGLU(176)]`
  - Predictor: `Linear(64, 7)`
  - Learning Rate: `0.001`
- **TS-I and TS-E Benchmarks:**
  - Embedder:
    - $2DS_\alpha$: `Linear(2, 32) + CTE(t)`[6]
    - USH: `Linear(5 , 64)` → `ReLU()` → `LN()` → $2 \times$ `[Linear(64, 64)` → `ReLU()` → `LN()]` → `Linear(64, 64)`
    - PHY: `Linear(37, 64)` → `ReLU()` → `LN()` → $2 \times$ `[Linear(64, 64)` → `ReLU()` → `LN()]` → `Linear(64, 64)`
  - Liquid Mixer:
    - $2DS_\alpha$: $1 \times$ `[MHLGA(32, 4)` → `SwiGLU(88)]`
    - USH and PHY: $2 \times$ `[MHLGA(64, 4)` → `SwiGLU(176)]`
  - Predictor:
    - $2DS_\alpha$: `Linear(32, 2)`
    - USH: $3 \times$ `[Linear(64, 64)`→`ReLU()]` → `Linear(64, 5)`
    - PHY: $3 \times$ `[Linear(64, 64)`→`ReLU()]` → `Linear(64, 37)`
  - Learning Rate: `0.01`
    - $2DS_\alpha$: `0.01`
    - USH and PHY: `0.001`

## REFERENCES


[1] J. A. Quinn, C. K. I. Williams, and N. McIntosh, "Factorial switching linear dynamical systems applied to physiological condition monitoring," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 31, no. 9, pp. 1537–1551, 2009.

[2] Z. Li, R. Cai, T. Z. J. Fu, Z. Hao, and K. Zhang, "Transferable time-series forecasting under causal conditional shift," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 4, pp. 1932–1949, 2024.


[6]Following [10], we incorporate a continuous time embedding (CTE) into the embedder to provide an auxiliary temporal signal alongside the liquid gate mechanism. The continuous time embedding replaces the step indices in the standard sinusoidal positional encoding [13] with observed timestamps.


[3] A. Elnaggar, M. Heinzinger, C. Dallago, G. Rehawi, Y. Wang, L. Jones, T. Gibbs, T. Feher, C. Angerer, M. Steinegger, D. Bhowmik, and B. Rost, "Prottrans: Toward understanding the language of life through self-supervised learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 10, pp. 7112–7127, 2022.

[4] S. N. Shukla and B. M. Marlin, "A survey on principles, models and methods for learning from irregularly sampled time series: From discretization to attention and invariance," *CoRR*, vol. abs/2012.00168, 2020.

[5] Y. Rubanova, T. Q. Chen, and D. Duvenaud, "Latent ordinary differential equations for irregularly-sampled time series," in *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, 2019, pp. 5321–5331.

[6] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *CoRR*, vol. abs/2312.00752, 2023.

[7] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Thirty-Third Conference on Innovative Applications of Artificial Intelligence, IAAI 2021, The Eleventh Symposium on Educational Advances in Artificial Intelligence, EAAI 2021, Virtual Event, February 2-9, 2021*, 2021, pp. 11 106–11 115.

[8] P. Kidger, "On neural differential equations," *arXiv preprint arXiv:2202.02435*, 2022.

[9] M. Schirmer, M. Eltayeb, S. Lessmann, and M. Rudolph, "Modeling irregular time series with continuous recurrent units," in *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, 2022, pp. 19 388–19 405.

[10] Y. Chen, K. Ren, Y. Wang, Y. Fang, W. Sun, and D. Li, "Contiformer: Continuous-time transformer for irregular time series modeling," in *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023.

[11] K. Cho, B. van Merrienboer, Ç. Gülçehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, October 25-29, 2014, Doha, Qatar, A meeting of SIGDAT, a Special Interest Group of the ACL*, 2014, pp. 1724–1734.

[12] S. Hochreiter and J. Schmidhuber, "Long long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, 2017, pp. 5998–6008.

[14] Z. C. Lipton, D. C. Kale, and R. C. Wetzel, "Directly modeling missing data in sequences with rnns: Improved classification of clinical time series," in *Proceedings of the 1st Machine Learning for Healthcare Conference*, 2016, pp. 253–270.

[15] M. Smieja, L. Struski, J. Tabor, B. Zielinski, and P. Spurek, "Processing of missing data by neural networks," in *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, 2018, pp. 2724–2734.

[16] E. Wang, Y. Jiang, Y. Xu, L. Wang, and Y. Yang, "Spatial-temporal interval aware sequential POI recommendation," in *38th IEEE International Conference on Data Engineering, ICDE 2022, Kuala Lumpur, Malaysia, May 9-13, 2022*, 2022, pp. 2086–2098.

[17] Y. Jiang, Y. Yang, Y. Xu, and E. Wang, "Spatial-temporal interval aware individual future trajectory prediction," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 10, pp. 5374–5387, 2024.

[18] D. Xu, C. Ruan, E. Korpeoglu, S. Kumar, and K. Achan, "Self-attention with functional time representation learning," in *Advances in Neural Information Processing Systems*, 2019, pp. 15 889–15 899.

[19] T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, "Neural ordinary differential equations," in *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, 2018, pp. 6572–6583.

[20] P. Kidger, J. Morrill, J. Foster, and T. J. Lyons, "Neural controlled differential equations for irregular time series," in *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020.

[21] J. Morrill, C. Salvi, P. Kidger, and J. Foster, "Neural rough differential equations for long time series," in *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, 2021, pp. 7829–7838.

[22] R. M. Hasani, M. Lechner, A. Amini, L. Liebenwein, A. Ray, M. Tschaikowski, G. Teschl, and D. Rus, "Closed-form continuous-time neural networks," *Nat. Mac. Intell.*, vol. 4, no. 11, pp. 992–1003, 2022.

[23] I. Kuleshov, E. Romanenkova, V. A. Zhuzhel, G. Boeva, E. Vorsin, and A. Zaytsev, "DeNOTS: Stable deep neural ODEs for time series," in *The Fourteenth International Conference on Learning Representations*, 2026.

[24] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces," in *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*, 2022.

[25] F. Moreno-Pino, A. Arroyo, H. Waldon, X. Dong, and A. Cartea, "Rough transformers: lightweight and continuous time series modelling through signature patching," in *Proceedings of the 38th International Conference on Neural Information Processing Systems*, 2024.

[26] R. M. Hasani, M. Lechner, A. Amini, D. Rus, and R. Grosu, "Liquid time-constant networks," in *Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Thirty-Third Conference on Innovative Applications of Artificial Intelligence, IAAI 2021, The Eleventh Symposium on Educational Advances in Artificial Intelligence, EAAI 2021, Virtual Event, February 2-9, 2021*, 2021, pp. 7657–7666.

[27] J. Schmidhuber, "Learning to control fast-weight memories: An alternative to dynamic recurrent networks," *Neural Comput.*, vol. 4, no. 1, pp. 131–139, 1992.

[28] M. C. Mozer, "Induction of multiscale temporal structure," in *Proceedings of the 5th International Conference on Neural Information Processing Systems*, 1991, pp. 275–282.

[29] E. Choi, M. T. Bahadori, A. Schuetz, W. F. Stewart, and J. Sun, "Doctor ai: Predicting clinical events via recurrent neural networks," in *Proceedings of the 1st Machine Learning for Healthcare Conference*, 2016, pp. 301–318.

[30] B. M. Marlin, D. C. Kale, R. G. Khemani, and R. C. Wetzel, "Unsupervised pattern discovery in electronic health care data using probabilistic clustering models," in *ACM International Health Informatics Symposium*, 2012, pp. 389–398.

[31] R. Little and D. Rubin, *Statistical Analysis with Missing Data*, 2014.

[32] Z. Che, S. Purushotham, K. Cho, D. Sontag, and Y. Liu, "Recurrent neural networks for multivariate time series with missing values," *Scientific Reports*, vol. 8, no. 1, p. 6085, 2018.

[33] J. Yoon, J. Jordon, and M. van der Schaar, "GAIN: missing data imputation using generative adversarial nets," in *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, July 10-15, 2018, Stockholmsmässan, Stockholm, Sweden*, 2018, pp. 5675–5684.

[34] M. Cheng, Z. Liu, X. Tao, Q. Liu, J. Zhang, T. Pan, S. Zhang, P. He, X. Zhang, D. Wang *et al.*, "A comprehensive survey of time series forecasting: Concepts, challenges, and future directions," *Authorea Preprints*, 2025.

[35] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *The Eleventh International Conference on Learning Representations, ICLR 2024, Kigali, Rwanda, May 1-5, 2024*, 2023.

[36] Q. Huang, L. Shen, R. Zhang, J. Cheng, S. Ding, Z. Zhou, and Y. Wang, "Hdmixer: Hierarchical dependency with extendable patch for multivariate time series forecasting," in *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, The Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2024, Vancouver, BC, Canada, February 20-27, 2024*, 2024, pp. 12 608–12 616.

[37] Y. Zhang, L. Ma, S. Pal, Y. Zhang, and M. Coates, "Multi-resolution time-series transformer for long-term forecasting," in *Proceedings of The 27th International Conference on Artificial Intelligence and Statistics*, 2024, pp. 4222–4230.

[38] V. Ekambaram, A. Jati, N. Nguyen, P. Sinthong, and J. Kalagnanam, "Tsmixer: Lightweight mlp-mixer model for multivariate time series forecasting," in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2023, Long Beach, CA, USA, August 6-10, 2023*, 2023, pp. 459–469.

[39] D. Luo and X. Wang, "Moderntcn: A modern pure convolution structure for general time series analysis," in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*, 2024.

[40] Y. Ma, Z. Liu, C. Zhuang, Y. Tan, Y. Dong, W. Zhong, and J. Gu, "Non-stationary time-aware kernelized attention for temporal event prediction," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Washington, DC, USA, August 14-18, 2022*, 2022, pp. 1224–1232.

[41] S. N. Shukla and B. M. Marlin, "Multi-time attention networks for irregularly sampled time series," in *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*, 2021.

[42] X. Li, T. L. Wong, R. T. Q. Chen, and D. Duvenaud, "Scalable gradients for stochastic differential equations," in *The 23rd International Conference on Artificial Intelligence and Statistics, AISTATS 2020, 26-28 August 2020, Online [Palermo, Sicily, Italy]*, ser. Proceedings of Machine Learning Research, S. Chiappa and R. Calandra, Eds. PMLR, 2020, pp. 3870–3882.

[43] J. Chien and Y. Chen, "Learning continuous-time dynamics with attention," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 2, pp. 1906–1918, 2023.

[44] Y. H. Lim, Q. Zhu, J. Selfridge, and M. F. Kasim, "Parallelizing non-linear sequential models over the sequence length," in *The Twelfth International Conference on Learning Representations*, 2024.

[45] B. Park, H. Lee, and J. Lee, "Amortized control of continuous state space feynman-kac model for irregular time series," in *The Thirteenth International Conference on Learning Representations*, 2025.

[46] J. Zhao, F. Chu, L. Xie, Y. Che, Y. Wu, and A. F. Burke, "A survey of transformer networks for time series forecasting," *Comput. Sci. Rev.*, vol. 60, p. 100883, 2026.

[47] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret, "Transformers are rnns: Fast autoregressive transformers with linear attention," in *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, vol. 119, 2020, pp. 5156–5165.

[48] I. Schlag, K. Irie, and J. Schmidhuber, "Linear transformers are secretly fast weight programmers," in *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, 2021, pp. 9355–9366.

[49] Y. Sun, L. Dong, S. Huang, S. Ma, Y. Xia, J. Xue, J. Wang, and F. Wei, "Retentive network: A successor to transformer for large language models," *CoRR*, vol. abs/2307.08621, 2023.

[50] S. Yang, B. Wang, Y. Shen, R. Panda, and Y. Kim, "Gated linear attention transformers with hardware-efficient training," in *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*, 2024, pp. 56 501–56 523.

[51] B. Peng, E. Alcaide, Q. Anthony, A. Albalak, S. Arcadinho, S. Biderman, H. Cao, X. Cheng, M. Chung, L. Derczynski, X. Du, M. Grella, K. K. GV, X. He, H. Hou, P. Kazienko, J. Kocon, J. Kong, J. L. Bartlomiej Koptyra Glen, Hayden Lau, K. S. I. Mantri, F. Mom, A. Saito, G. Song, X. Tang, J. S. Wind, S. Wozniak, Z. Zhang, Q. Zhou, J. Zhu, and R. Zhu, "RWKV: reinventing rnns for the transformer era," in *Findings of the Association for Computational Linguistics: EMNLP 2023, Singapore, December 6-10, 2023*, 2023, pp. 14 048–14 077.

[52] B. Peng, D. Goldstein, Q. Anthony, A. Albalak, E. Alcaide, S. Biderman, E. Cheah, X. D. triumphs, T. Ferdinan, H. Hou, P. Kazienko, K. K. GV, J. Kocon, B. Koptyra, S. Krishna, R. M. Jr., N. Muennighoff, F. Obeid, A. Saito, G. Song, H. Tu, S. Wozniak, R. Zhang, B. Zhao, Q. Zhao, P. Zhou, J. Zhu, and R. Zhu, "Eagle and finch: RWKV with matrix-valued states and dynamic recurrence," *CoRR*, vol. abs/2404.05892, 2024.

[53] B. Peng, R. Zhang, D. Goldstein, H. H. Eric Alcaide Euphrat, Xingjian Du, J. Lin, J. Liu, J. Lu, W. Merrill, G. Song, K. Tan, S. Utpala, N. Wilce, J. S. Wind, T. Wu, D. Wuttke, and C. Zhou-Zheng, "RWKV-7 "goose" with expressive dynamic state evolution," *CoRR*, vol. abs/2503.14456, 2025.

[54] T. Dao and A. Gu, "Transformers are ssms: Generalized models and efficient algorithms through structured state space duality," in *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*, 2024.

[55] S. Yang, B. Wang, Y. Zhang, Y. Shen, and Y. Kim, "Parallelizing linear transformers with the delta rule over sequence length," in *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024.

[56] J. Zhang, R. Su, C. Liu, J. Wei, Z. Wang, P. Zhang, H. Wang, H. Jiang, H. Huang, C. Xiang *et al.*, "Efficient attention methods: Hardware-efficient, sparse, compact, and linear attention."

[57] L. Perko, *Differential Equations and Dynamical Systems*, 1991.

[58] M. Beck, K. Pöppel, M. Spanring, A. Auer, O. Prudnikova, M. Kopp, G. Klambauer, J. Brandstetter, and S. Hochreiter, "xlstm: Extended long short-term memory," in *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024.

[59] A. J. Bagnall, J. Lines, A. Bostrom, J. Large, and E. J. Keogh, "The great time series classification bake off: a review and experimental evaluation of recent algorithmic advances," *Data Min. Knowl. Discov.*, vol. 31, no. 3, pp. 606–660, 2017.

[60] M. A. F. Pimentel, A. E. W. Johnson, P. Charlton, D. A. Birrenkott, P. J. Watkinson, L. Tarassenko, and D. A. Clifton, "Toward a robust estimation of respiratory rate from pulse oximeters," *IEEE Trans. Biomed. Eng.*, vol. 64, no. 8, pp. 1914–1923, 2017.

[61] P. Becker, H. Pandya, G. H. W. Gebhardt, C. Zhao, C. J. Taylor, and G. N. Gerhmann, "Recurrent kalman networks: Factorized inference in high-dimensional deep feature spaces," in *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, 2019, pp. 544–552.

[62] M. Menne, C. Williams, Jr., and R. Vose, "Long-term daily and monthly climate records from stations across the contiguous united states (u.s. historical climatology network)," 01 2016.

[63] I. Silva, G. B. Moody, D. J. Scott, L. A. Celi, and R. G. Mark, "Predicting in-hospital mortality of icu patients: The physionet/computing in cardiology challenge 2012," *Computing in cardiology*, vol. 39, pp. 245 – 248, 2012. [Online]. Available: https://api.semanticscholar.org/CorpusID:8678934

[64] A. Orvieto, S. L. Smith, A. Gu, A. Fernando, C. Gulcehre, R. Pascanu, and S. De, "Resurrecting recurrent neural networks for long sequences," in *Proceedings of the 40th International Conference on Machine Learning*, 2023.

[65] J. Chung, Ç. Gülçehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," *CoRR*, vol. abs/1412.3555, 2014.

[66] S. Zeng, F. Graf, and R. Kwitt, "Latent sdes on homogeneous spaces," in *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023.

[67] B. Walker, A. D. McLeod, T. Qin, Y. Cheng, H. Li, and T. J. Lyons, "Log neural controlled differential equations: The lie brackets make A difference," in *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*, 2024, pp. 49 822–49 844.

[68] E. D. Brouwer, J. Simm, A. Arany, and Y. Moreau, "Gru-ode-bayes: Continuous modeling of sporadically-observed time series," in *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, H. M. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. B. Fox, and R. Garnett, Eds., 2019, pp. 7377–7388.

[69] S. N. Shukla and B. Marlin, "Multi-time attention networks for irregularly sampled time series," in *International Conference on Learning Representations*, 2021.

[70] J. T. Smith, A. Warrington, and S. Linderman, "Simplified state space layers for sequence modeling," in *The Eleventh International Conference on Learning Representations*, 2023.